may need in order to proceed, while the second process may hold resources needed by the first process—a two-process deadlock.

Havender's second strategy requires that when a process holding resources is denied a request for additional resources, it must release the resources it holds and, if necessary, request them again together with the additional resources. This strategy effectively denies the "no-preemption" condition—resources can indeed be removed from the process holding them prior to the completion of that process.

Here, too, the means for preventing deadlock can be costly. When a process releases resources, it may lose all of its work to that point. This may seem to be a high price to pay, but the real question is, "How often does this price have to be paid?" If this occurs infrequently, then this strategy provides a relatively low-cost means of preventing deadlocks. If it occurs frequently, however, then the cost is substantial and the effects are disruptive, particularly when high-priority or deadline processes cannot be completed on time because of repeated preemptions.

Could this strategy lead to indefinite postponement? It depends. If the system favors processes with small resource requests over those requesting substantial resources, that alone could lead to indefinite postponement. Worse yet, as the process requests additional resources, this Havender strategy requires the process to give up all the resources it has and request an even larger number. So indefinite postponement can be a problem in a busy system. Also, this strategy requires all resources to be preemptible, which is not always the case (e.g., printers should not be preempted while processing a print job).

Self Review

1. What is the primary cost of denying the "no-preemption" condition?
2. Which of Ha vender's first two deadlock strategies do you suppose people find more palatable? Why?

Ans: **1)** A process may lose all of its work up to the point that its resources were preempted. Also, the process could suffer indefinite postponement, depending on the system's resource-allocation strategy. **2)** Most people probably would prefer the first strategy, namely, requiring a process to request all the resources it will need in advance. The second strategy requires the process to give up the resources it already has, possibly causing wasteful loss of work. Interestingly, the first strategy could cause waste as well, as processes gradually acquire resources they cannot yet use.

7.7.3 Denying the "Circular-Wait" Condition

Havender's third strategy denies the possibility of a circular wait. In this strategy. we assign a unique number to each resource (e.g., a disk drive, printer, scanner, file) that the system manages and we create a **linear ordering** of resources. A process must then request its resources in a strictly ascending order. For example, if a process requests resource R3 (where the subscript, 3, is the resource number), then the process can subsequently request only resources with a number greater than 3.

Because all resources are uniquely numbered, and because processes must request resources in ascending order, a circular wait cannot develop (Fig. 7.5). *[Note: A* proof of of this property is straightforward. Consider a system that enforces a linear ordering of resources in which R_i and R_j are resources numbered by integers i and j, respectively $(i \neq j)$. If the system has a circular wait characteristic of deadlock, then according to Fig. 7.5, at least one arrow (or set of arrows) leads upward from R_i to R_j and an arrow (or set of arrows) exists leading down from R_j to R_i. However, the linear ordering of resources with the requirement that resources be requested in ascending order implies that no arrow can ever lead from R_j to R_i if $j > i$, Therefore deadlock cannot occur in this system.]

Denying the "circular-wait" condition has been implemented in a number of legacy operating systems, but not without difficulties.[15, 16, 17, 18] One disadvantage of this strategy is that it is not as flexible or dynamic as we might desire. Resources must be requested in ascending order by resource number. Resource numbers are assigned for the computer system and must be "lived with" for long periods (i.e., months or even years). If new resources are added or old ones removed at an installation, existing programs and systems may have to be rewritten.

Another difficulty is determining the ordering of resources in a system. Clearly, the resource numbers should be assigned to reflect the order in which most processes actually use the resources. For processes matching this ordering, more efficient operation may be expected. But for processes that need the resources in a different order than that specified by the linear ordering, resources must be acquired and held, possibly for long periods of time, before they are actually used. This can result in poor performance.

An important goal in today's operating systems is to promote software portability across multiple environments. Programmers should be able to develop their applications without being impeded by awkward hardware and software restrictions. Havender's linear ordering truly eliminates the possibility of a circular wait, yet it diminishes a programmer's ability to freely and easily write application code that will maximize an application's performance.

Self Review

1. How does a linear ordering for resource allocation reduce application portability?
2. (T/F) Imposing a linear ordering for resource requests yields higher performance than denying the "no-preemption" condition.

Ans: **1)** Different systems will normally have different sets of resources and may order resources differently, so an application written for one system may need to be modified to run effectively on another. **2)** False. There are situations where each solution results in higher performance because the solution requires insignificant overhead. If a system's processes each request disjoint sets of resources, denying "no preemption" is quite efficient. If a process uses resources in an order corresponding to the system's linear ordering, then denying the "circular wait" condition can result in higher performance.

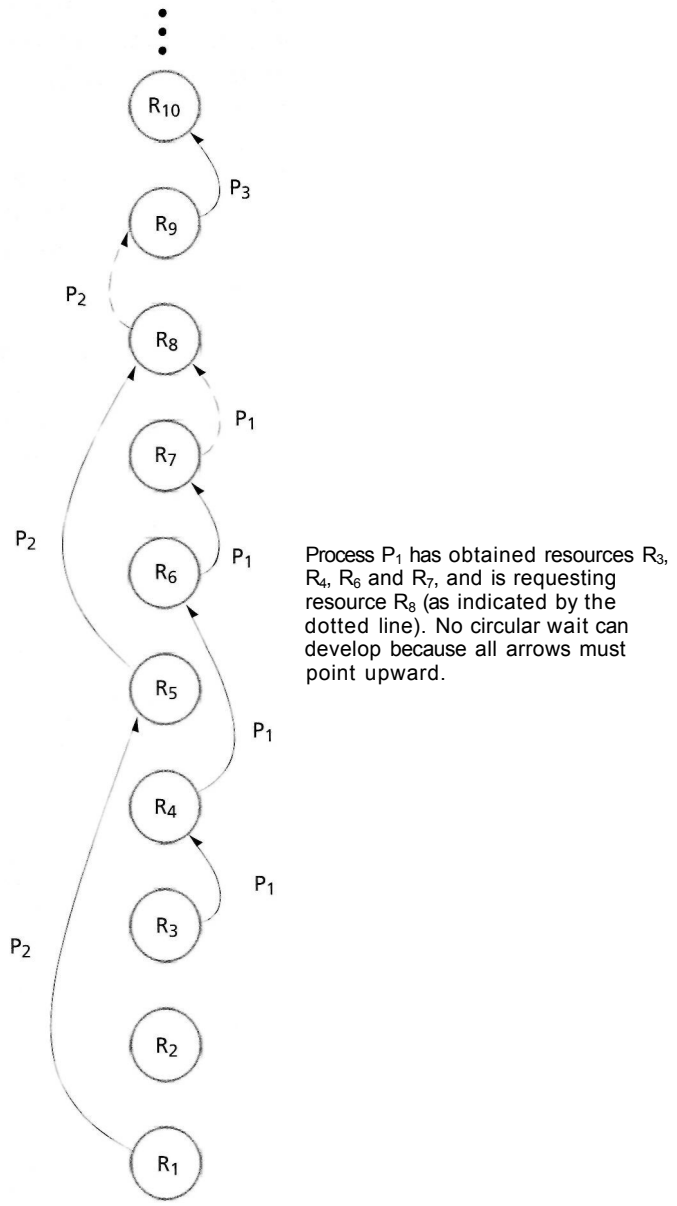

Figure 7.5 | *Havender's linear ordering of of resources for preventing deadlock.*

7.8 Deadlock Avoidance with Dijkstra's Banker's Algorithm

For some systems, it is impractical to implement the deadlock prevention strategies we discussed in the preceding section. However, if the necessary conditions for a

deadlock to occur are in place, it is still possible to avoid deadlock by carefully allo--ting system resources. Perhaps the most famous deadlock avoidance algorithm is **Dijkstra's Banker's Algorithm,** so named because the strategy is modeled after a banker who makes loans from a pool of capital and receives payments that are returned to that pool.[19, 20, 21, 22, 23] We paraphrase the algorithm here in the context of operating systems resource allocation. Subsequently, much work has been done in deadlock avoidance.[24, 25, 26, 27, 28, 29, 30, 31]

The Banker's Algorithm defines how a particular system can prevent deadlock by carefully controlling how resources are distributed to users (see the Anecdote, Acronyms). A system groups all the resources it manages into **resource types.** Each resource type corresponds to resources that provide identical functionality. To simplify our presentation of the Banker's Algorithm, we limit our discussion to a system that manages only one type of resource. The algorithm is easily extendable to pools of resources of various types; this is left to the exercises.

The Banker's Algorithm prevents deadlock in operating systems that exhibit the following properties:

- The operating system shares a fixed number of resources, *t,* among a fixed number of processes, *n.*
- Each process specifies in advance the maximum number of resources that it requires to complete its work.
- The operating system accepts a process's request if that process's **maximum need** does not exceed the total number of resources available in the system, *t* (i.e., the process cannot request more than the total number of resources available in the system).
- Sometimes, a process may have to wait to obtain an additional resource, but the operating system guarantees a finite wait time.
- If the operating system is able to satisfy a process's maximum need for resources, then the process guarantees that the resource will be used and released to the operating system within a finite time.

Anecdote

Acronyms

Most computer folks are familiar with the acronym WYSIWYG for "what you see is what you get." Few people have seen these two-IWWIWWIWI and YGWIGWIGI, which nicely describe the relationship between the process requesting resources and the operating system attempting to allocate them. The process says, "I want what I want when I want it." The operating system responds, "You'll get what I've got when I get it."

The system is said to be in a safe **state** if the operating system can guarantee that all current processes can complete their work within a finite time. If not, then the system is said to be in an **unsafe state.**

We also define four terms that describe the distribution of resources among processes.

- Let $max(P_i)$ be the maximum number of resources that process P_i requires during its execution. For example, if process P_3 never requires more than two resources, then $max(P_3) = 2$.
- Let $loan(P_i)$ represent process P_i's current **loan** of a resource, where its loan is the number of resources the process has already obtained from the system. For example, if the system has allocated four resources to process P_5, then $loan(P_5) = 4$.
- Let $claim(P_i)$ be the current **claim** of a process, where a process's claim is equal to its maximum need minus its current loan. For example, if process P_7 has a maximum need of six resources and a current loan of four resources, then we have

  $$claim(P_7) = max(P_7) - loan(P_7) = 6 - 4 = 2$$

- Let a be the number of resources still available for allocation. This is equivalent to the total number of resources (t) minus the sum of the loans to all the processes in the system, i.e.,

  $$a = t - \sum_{i=1}^{n} loan(P_i)$$

So, if the system has a total of three processes and 12 resources, and the system has allocated two resources to process P_1, one resource to process P_2 and four resources to process P3, then the number of available resources is

$$a = 12 - (2 + 1 + 4) = 12 - 7 = 5$$

Dijkstra's Banker's Algorithm requires that resources be allocated to processes only when the allocations result in safe states. In that way, all processes remain in safe states at all times, and the system will never deadlock.

Self Review

1. (T/F) An unsafe state is a deadlocked state.
2. Describe the restrictions that the Banker's Algorithm places on processes.

Ans: **1)** False. A process in an unsafe state might eventually deadlock, or it might complete its execution without entering deadlock. What makes the state unsafe is simply that the operating system cannot guarantee that from this state all processes can complete their work. From an unsafe state, it is possible but not guaranteed that all processes could complete their work, so a system in an unsafe state could eventually deadlock. **2)** Each process, before it runs, is required to specify the maximum number of resources it may require at any point during its execution. Each process cannot request more than the total number of resources in

the system. Each process must also guarantee that once allocated a resource, the process will eventually return that resource to the system within a finite time.

7.8.1 Example of a Safe State

Suppose a system contains 12 equivalent resources and three processes sharing the resources, as in Fig. 7.6. The second column contains the maximum need for each process, the third column the current loan for each process and the fourth column the current claim for each process. The number of available resources, *a,* is two; this is computed by subtracting the sum of the current claims from the total number of resources, $t = 12$.

This state is "safe" because it is possible for all three processes to finish. Note that process P_2 currently has a loan of four resources and will eventually need a maximum of six, or two additional resources. The system has 12 resources, of which 10 are currently in use and two are available. If the system allocates these two available resources to P_2, fulfilling P_2's maximum need, then P_2 can run to completion. Note that after P_2 finishes, it will release six resources, enabling the system to immediately fulfill the maximum needs of P_1 (3) and P3 (3), enabling both of those processes to finish. Indeed, all processes can eventually finish from the safe sate of Fig. 7.6.

Self Review

1. In Fig. 7.6, once P_2 runs to completion, must P_1 and P_3 run sequentially one after the other, or could they run concurrently?
2. Why did we focus initially on allowing P_2 to finish rather than focusing on either P_1 or P_3?

Ans: **1)** P_1 and P_3 each need three more resources to complete. When P_2 finishes, it will release six resources, which is enough to allow P_1 and P_3 to run concurrently. **2)** Because P_2 is the only process whose current claim can be satisfied by the two available resources.

7.8.2 Example of an Unsafe State

Assume a system's 12 resources are allocated as in Fig. 7.7. We sum the values of the third column and subtract from 12 to obtain a value of one for *a.* At this point, no matter which process requests the available resource, we cannot guarantee that all three processes will finish. In fact, suppose process P_1 requests and is granted the last available resource. A three-way deadlock could occur if indeed each process

Process	*max(P_i) (maximum need)*	***loan (P_i) (current loan)***	*claim(P_i) (current claim)*
P_1	**4**	**1**	**3**
P_2	**6**	**4**	**2**
P_3	**8**	**5**	**3**
Total resources, X, = 12		*Available resources,* a, *= 2*	

Figure 7.6 | Safe State.

Process	max(P_i) (maximum need)	loan(P_i) (current loan)	claim (P_i) (current claim)
P_1	10	8	2
p_2	5	2	3
p_3	3	1	2
Total resources, t, = 12		Available resources, a, = 1	

Figure 7.7 | Unsafe state.

needs to request at least one more resource before releasing any resources to the pool. It is important to note here that an *unsafe state does not imply the existence of deadlock, nor even that deadlock will eventually occur. What an unsafe state does imply is simply that some unfortunate sequence of events might lead to a deadlock.*

1. Why is deadlock possible, but not guaranteed, when a system enters an unsafe state?
2. What minimum number of resources would have to be added to the system of Fig. 7.7 to make the state safe?

Ans: **1)** Processes could give back their resources early, increasing the number of available resources to the point that the state of the system was once again safe and all other processes could finish. **2)** By adding one resource, the number of available resources becomes two, enabling P_1 to finish and return its resources, enabling both P_2 and P_3 to finish. Hence the new state is safe.

7.8.3 Example of Safe-State-to-Unsafe-State Transition

Our resource-allocation policy must carefully consider all resource requests before granting them, or a process in a safe state could enter an unsafe state. For example, suppose the current state of a system is safe, as shown in Fig. 7.6. The current value of *a* is 2. Now suppose that process P3 requests an additional resource. If the system were to grant this request, then the new state would be as in Fig. 7.8. Now, the current value of *a* is 1, which is not enough to satisfy the current claim of any process, so the state is now unsafe.

Process	max(P_i) (maximum need)	loan(P_i) (current loan)	claim(P_i) (current claim)
P_1	4	1	3
P_2	6	4	2
P_3	8	6	2
Total resources, t, = 12		Available resources, a, = 1	

Figure 7.8 | *Safe-state-to-unsafe-state transition.*

Self Review

1. If process P_2 requested one additional resource in Fig. 7.6, would the system be in a safe or an unsafe state?
2. *(TIF)* A system cannot transition from an unsafe state to a safe state.

Ans: **1)** The system would still be in a safe state because there is a path of execution that will not result in deadlock. For example, if P_2 requests another resource, then P_2 can complete execution. Then there will be six available resources, which is enough for P_1 and P_3 to finish. **2)** False. As processes release resources, the number of available resources can become sufficiently large for the state of the system to transition from unsafe to safe.

7.8.4 Banker's Algorithm Resource Allocation

Now it should be clear how resource allocation operates under Dijkstra's Banker's Algorithm. The "mutual exclusion," "wait-for," and "no-preemption" conditions are allowed—processes are indeed allowed to hold resources while requesting and waiting for additional resources, and resources may not be preempted from a process holding those resources. As usual, processes claim exclusive use of the resources they require. Processes ease onto the system by requesting one resource at a time; the system may either grant or deny each request. If a request is denied, that process holds any allocated resources and waits for a finite time until that request is eventually granted. The system grants only requests that result in safe states. Resource requests that would result in unsafe states are repeatedly denied until they can eventually be satisfied. Because the system is always maintained in a safe state, sooner or later (i.e., in a finite time) all requests can be satisfied and all users can finish.

Self Review

1. (T/F) The state described by Figure 7.9 is safe.
2. What minimum number of additional resources, when added to the system of Fig. 7.9, would make the state safe?

Ans: **1)** False. There is no guarantee that all of these processes will finish. P_2 will be able to finish by using up the two remaining resources. However, once P_2 is done, there are only three available resources left. This is not enough to satisfy either P_1's claim of 4 or P_3's claim of five. **2)** By adding one more resource, we could allow P_2 to finish and return three resources. These plus the added resource would enable P_1 to finish, returning five resources and enabling P_3 to finish.

Process	*max(P_i)*	*loan(P_i)*	*claim(P_i)*
P_1	**5**	**1**	**4**
P_2	**3**	**1**	**2**
P_3	**10**	**5**	**5**
		a = 2	

Figure 7.9 | State description of three processes.

7.8.5 Weaknesses in the Banker's Algorithm

The Banker's Algorithm is compelling because it allows processes to proceed that might have had to wait under a deadlock prevention situation. But the algorithm has a number of weaknesses.

- It requires that there be a fixed number of resources to allocate. Because resources frequently require service, due to breakdowns or preventive maintenance, we cannot count on the number of resources remaining fixed. Similarly, operating systems that support hot swappable devices (e.g., USB devices) allow the number of resources to vary dynamically.
- The algorithm requires that the population of processes remains fixed. This, too, is unreasonable. In today's interactive and multiprogrammed systems. the process population is constantly changing.
- The algorithm requires that the banker (i.e., the system) grant all requests within a "finite time." Clearly, much better guarantees than this are needed in real systems, especially real-time systems.
- Similarly, the algorithm requires that clients (i.e., processes) repay all loans (i.e., return all resources) within a "finite time." Again, much better guarantees than this are needed in real systems.
- The algorithm requires that processes state their maximum needs in advance. With resource allocation becoming increasingly dynamic, it is becoming more difficult to know a process's maximum needs. Indeed, one main benefit of today's high-level programming languages and "friendly" graphical user interfaces is that users are not required to know such low-level details as resource use. The user or programmer expects the system to "print the file" or "send the message" and should not need to worry about what resources the system might need to employ to honor such requests.

For the reasons stated above, Dijkstra's Banker's Algorithm is not implemented in today's operating systems. In fact, few systems can afford the overhead incurred by deadlock avoidance strategies.

Self Review

1. Why does the Banker's Algorithm fail in systems that support hot swappable devices?

Ans: The Banker's Algorithm requires that the number of resources of each type remain fixed. Hot swappable devices can be added and removed from the system at any time, meaning that the number of resources of each type can vary.

7.9 Deadlock Detection

We have discussed deadlock prevention and avoidance—two strategies for ensuring that deadlocks do not occur in a system. Another strategy is to allow deadlocks to occur, locate them and remove them, if possible. **Deadlock detection** is the process of determining that a deadlock exists and identifying the processes and resources

involved in the deadlock.[32,33,34,35,36,37,38,39,40] Deadlock detection algorithms generally focus on determining if a circular wait exists, given that the other necessary conditions for deadlock are in place.

Deadlock detection algorithms can incur significant runtime overhead. Thus we again face the trade-offs so prevalent in operating systems—is the overhead involved in deadlock detection justified by the potential savings from locating and breaking up deadlocks? For the moment, we shall ignore this issue and concentrate instead on the development of algorithms capable of detecting deadlocks.

7.9.1 Resource-Allocation Graphs

To facilitate the detection of deadlocks, a popular notation (Fig. 7.10) is used in which a directed graph indicates resource allocations and requests.[41] Squares represent processes and large circles represent classes of identical resources. Small circles drawn inside large circles indicate the separate identical resources of each class. For example, a large circle labeled "R_1" containing three small circles indicates that there are three equivalent resources of type R_1 available for allocation in this system.

Figure 7.10 illustrates the relationships that may be indicated in a resource-allocation and request graph. In Fig. 7.10(a), process P_1 is requesting a resource of type R_1. The arrow from P_1 touches only the extremity of the large circle, indicating that the resource request is under consideration.

In Fig. 7.10(b), process P_2 has been allocated a resource of type R_2 (of which there are two).The arrow is drawn from the small circle within the large circle R_2 to the square P_2, to indicate that the system has allocated a specific resource of that type to the process.

Figure 7.10(c) indicates a situation somewhat closer to a potential deadlock. Process P_3 is requesting a resource of type R_3, but the system has allocated the only R_3 resource to process P_4.

Figure 7.10(d) indicates a deadlocked system in which process P_5 is requesting a resource of type R_4, the only one of which the system has allocated to process P_6. Process P_6, is requesting a resource of type R_5, the only one of which the system has allocated to process P_5. This is an example of the "circular wait" necessary for a deadlocked system.

Resource-allocation and request graphs change as processes request resources, acquire them and eventually release them to the operating system.

Self Review

1. Suppose a process has control of a resource of type R_1. Does it matter which small circle points to the process in the resource-allocation graph?
2. What necessary condition for deadlock is easier to identify in a resource-allocation graph than it is to locate by analyzing the resource-allocation data of all the system's processes?

Ans: **1)** No; all resources of the same type must provide identical functionality, so it does not matter which small circle within the circle R_1 points to the process. **2)** Resource-allocation graphs make it easier to identify circular waits.

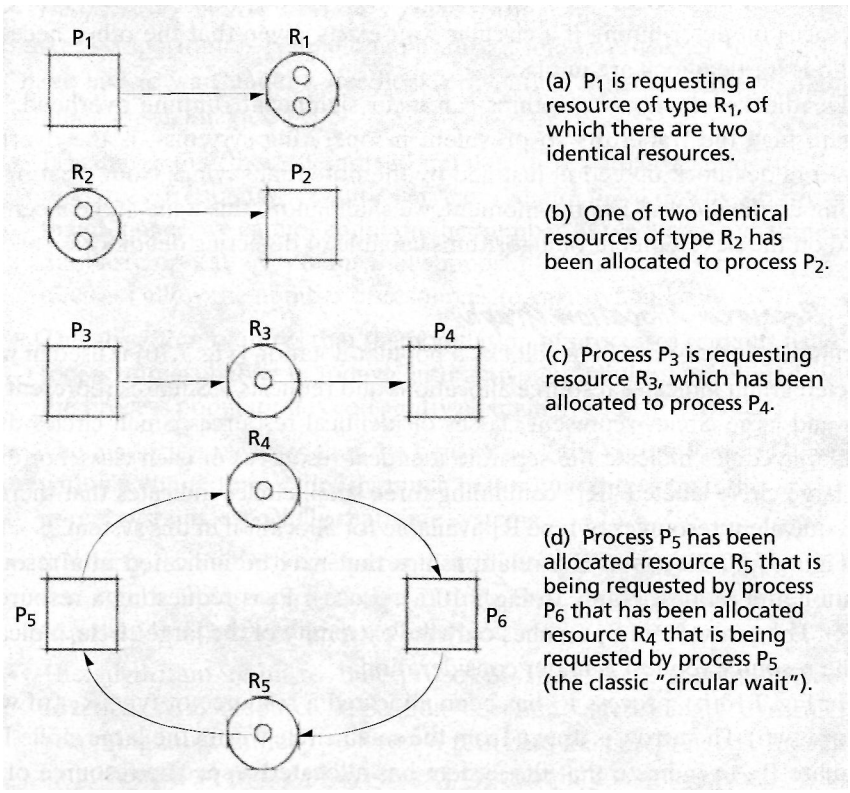

Figure 7.10 | Resource-allocation and request graphs.

7.9.2 Reduction of Resource-Allocation Graphs

One technique useful for detecting deadlocks involves graph reductions, in which the processes that may complete their execution, if any, and the processes that will remain deadlocked (and the resources involved in that deadlock), if any, are determined.[42]

If a process's resource requests may be granted, then we say that a graph may be **reduced** by that process. This reduction is equivalent to showing how the graph would look if the process was allowed to complete its execution and return its resources to the system. We reduce a graph by a process by removing the arrows to that process from resources (i.e., resources allocated to that process) and by removing arrows from that process to resources (i.e., current resource requests of that process). If a graph can be reduced by all its processes, then there is no deadlock. If a graph cannot be reduced by all its processes, then the irreducible processes constitute the set of deadlocked processes in the graph.

Figure 7.11 shows a series of graph reductions demonstrating that a particular set of processes is not deadlocked. Figure 7.10(d) shows an irreducible set of processes that constitutes a deadlocked system. It is important to note here that the order in

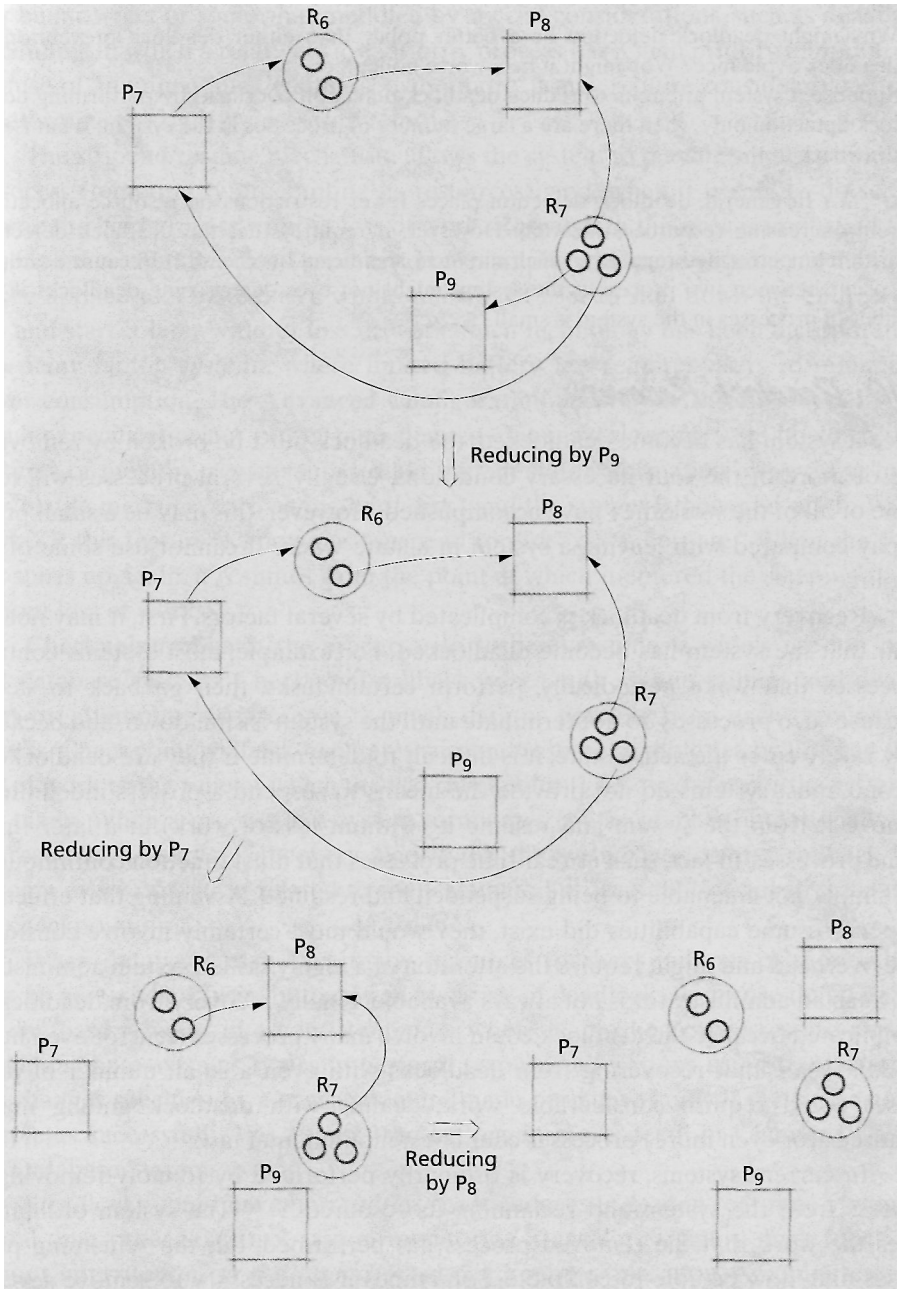

Figure 7.11 | *Graph reductions determining that no deadlock exists*

which the graph reductions are performed does not matter: The final result will always be the same. We leave the proof of this result to the exercises (see Exercise 7.29).

Self Review

1. Why might deadlock detection be a better policy than either deadlock prevention or deadlock avoidance? Why might it be a worse policy?
2. Suppose a system attempts to reduce deadlock detection overhead by performing deadlock detection only when there are a large number of processes in the system. What is one drawback to this strategy?

Ans: **1)** In general, deadlock detection places fewer restrictions on resource allocation, thereby increasing resource utilization. However, it requires that the deadlock detection algorithm be performed regularly, which can incur significant overhead. **2)** Because deadlock can occur between two processes, the system might not ever detect some deadlocks if the number of processes in the system is small.

7.10 Deadlock Recovery

Once a system has become deadlocked, the deadlock must be broken by removing one or more of the four necessary conditions. Usually, several processes will lose some or all of the work they have accomplished. However, this may be a small price to pay compared with leaving a system in a state where it cannot use some of its resources.

Recovery from deadlock is complicated by several factors. First, it may not be clear that the system has become deadlocked. For example, most systems contain processes that wake periodically, perform certain tasks, then go back to sleep. Because such processes do not terminate until the system is shut down, and because they rarely enter the active state, it is difficult to determine if they are deadlocked. Second, most systems do not provide the means to suspend a process indefinitely, remove it from the system and resume it (without loss of work) at a later time. Some processes, in fact, such as real-time processes that must function continuously, are simply not amenable to being suspended and resumed. Assuming that effective suspend/resume capabilities did exist, they would most certainly involve considerable overhead and might require the attention of a highly skilled system administrator. Such an administrator is not always available. Finally, recovery from deadlock is complicated because the deadlock could involve many processes (tens, or even hundreds). Given that recovering from deadlocks with even a small number of processes could require considerable work, dealing with deadlock among many hundreds (or even more) processes could be a monumental task.

In current systems, recovery is ordinarily performed by forcibly removing a process from the system and reclaiming its resources.[43, 44] The system ordinarily loses the work that the removed process has performed, but the remaining processes may now be able to complete. Sometimes it is necessary to remove several processes until sufficient resources have been reclaimed to allow the remaining processes to finish. Recovery somehow seems like an inappropriate term here, because some processes are in fact "killed" for the benefit of the others.

Processes may be removed according to some priority order. Here, too, we face several difficulties. For example, the priorities of the deadlocked processes may

not exist, so the system may need to make an arbitrary decision. The priorities also may be incorrect or somewhat muddled by special considerations, such as **deadline scheduling**, in which a relatively low-priority process has a high priority temporarily because of an impending deadline. Furthermore, it may require considerable effort to determine the right processes to remove.

The suspend/resume mechanism allows the system to put a temporary hold on a process (temporarily preempting its resources), and, when it is safe to do so, to resume the held process without loss of work. Research in this area is important for reasons other than deadlock recovery. For example, a suspend/resume mechanism can be applied to a system as a whole, allowing a user to shut down the entire system and start it later without loss of work. Such technology has been incorporated into many laptop systems, where limited battery life requires users to minimize power consumption. The **Advanced Configuration and Power Interface (ACPI),** a popular specification for power management, defines a sleeping state in which the contents of memory, registers and other system state information are written to a nonvolatile medium (such as the hard disk) and the system is powered off. In Windows XP, this feature is known as "suspend to disk" or "hibernate." When the system starts up again, it resumes from the point at which it entered the sleeping state without loss of work.[45]

Checkpoint/rollback, the precursor to suspend/resume, is widely used in current database systems. Checkpoint/rollback copes with system failures and deadlocks by attempting to preserve as much data as possible from each terminated process. Checkpoint/rollback facilitates suspend/resume capabilities by limiting the loss of work to the time at which the last **checkpoint** (i.e., saved state of the system) was taken. When a process in a system terminates (by accident or intentionally as the result of a deadlock recovery algorithm), the system performs a **rollback** by undoing every operation related to the terminated process that occurred since the last checkpoint.

When databases may have many resources (perhaps millions or more) that must be accessed exclusively, there can be a risk of deadlock. To ensure that data in the database remains in a consistent state when deadlocked processes are terminated, database systems typically perform resource allocations using **transactions.** The changes specified by a transaction are made permanent only if the transaction completes successfully. We discuss transactions in more detail in Chapter 13, File and Database Systems.

Deadlocks could have horrendous consequences in certain real-time systems. A real-time process control system monitoring a gasoline refinery must function without interruption for the refinery's safe and proper operation. A computerized heart pacemaker must literally not "miss a beat." Deadlocks cannot be risked in such environments. What would happen if a deadlock did develop? Clearly, it would have to be detected and removed instantaneously. But is this always possible? These are some of the considerations that keep operating systems designers from restful sleep.

Self Review

1. (T/F) A system can eliminate deadlock by choosing at random an irreducible process in a resource-allocation graph.
2. Why might a system that restarts a process it "kills" to break a deadlock suffer from poor performance?

Ans: **1)** False. There may be multiple circular waits within the system by the time a deadlock is detected. **2)** First, because killing a process causes loss of work. Second, because the restarted process will execute the same code that caused the initial deadlock, and if the system state has not changed, the system can become deadlocked repeatedly.

7.11 Deadlock Strategies in Current and Future Systems

In personal computer systems and workstations, deadlock has generally been viewed as a limited annoyance. Some systems implement the basic deadlock prevention methods suggested by Havender while others ignore the problem—these methods seem to be satisfactory. While ignoring deadlocks may seem dangerous, this approach can actually be rather efficient. Consider that today's systems can contain thousands or millions of serially reusable objects on which processes and threads can deadlock. The time required to execute a deadlock detection algorithm can rise exponentially with the number of serially reusable objects in the system. If deadlock is rare, then the processor time devoted to checking for deadlocks significantly reduces system performance. In systems that are neither mission critical nor business critical, the choice of ignoring deadlock to favor performance often outweighs the need for a solution to the occasional deadlock.

Although some systems ignore deadlock that occurs due to user processes, it is far more important to prevent deadlock in the operating system. Systems such as Microsoft Windows provide debugger support, allowing developers to thoroughly test drivers and applications to ensure that they acquire resources without causing deadlock (e.g., they do not attempt to acquire locks in recursive routines, or they specify lock acquisition in a certain order).[46] Interestingly, once such programs are released, these testing mechanisms are often disabled to improve efficiency.[47]

In real-time, mission-critical or business-critical systems the possibility of deadlock cannot be tolerated. Researchers have developed techniques that handle deadlock while minimizing data loss and maintaining good performance. For example, deadlock is commonly addressed in distributed database systems, which could provide concurrent access to billions of records for millions of users over thousands of sites.[48] Due to the large size of distributed database systems, they often do not employ deadlock prevention or deadlock avoidance algorithms. Instead, they rely on deadlock detection and recovery via checkpoint/rollback (using transactions).[49] These techniques are beyond the scope of this chapter; see Chapter 17, Introduction to Distributed Systems, for an introduction to such methods.

Given current trends, deadlock will continue to be an important area of research for several reasons:

- Many large-scale systems are oriented more toward asynchronous parallel operations than toward the serial operations of the past. Multiprocessing is common, and parallel computation will be prevalent. Networks and distributed systems are ubiquitous. There are, quite simply, more operations going on concurrently, more conflicts over resources and hence more chances of deadlock. Consequently, research on deadlock detection and recovery in distributed systems has become quite active.
- With the increasing tendency of operating systems designers to view data as a resource, the number of resources that operating systems must manage is increasing dramatically. This is particularly evident in Web servers and database systems, which require high resource utilization and performance. This makes most deadlock prevention techniques impractical and deadlock recovery algorithms more important. Researchers have developed advanced transaction-based algorithms that ensure high resource utilization while maintaining a low cost of deadlock recovery.[50]
- Hundreds of millions of computers are incorporated into common devices, particularly small, portable ones such as cell phones, PDAs and navigation systems. These systems, increasingly characterized as Systems-on-a-Chip (SoC), are limited by a small set of resources and the demands of real-time tasks.[51, 52] Deadlock-free resource allocation in such systems is essential, because users cannot rely on an administrator to detect and rid the system of the deadlock.

Self Review

1. Why is deadlock prevention not a primary concern for many operating systems?
2. Why might deadlock in distributed systems be more difficult to detect than in a single computer?

Ans: **1)** Deadlock is rare and is often considered a minor annoyance for most personal computer users, who are often more concerned with operating system performance and features. **2)** Deadlock in distributed systems can be difficult to detect because each computer is managed by a different operating system, requiring each operating system to collect information from other computers in order to construct its resource-allocation graph.

Web Resources

www.diee.unica.it/~giua/PAPERS/CONF/03smc_b.pdf
Discusses an efficient approach to deadlock prevention in a real-time system.

www.db.fmi.uni-passau.de/publications/techreports/dda.html
Describes a solution to deadlock detection in distributed systems.

www.linux-mag.com/2001-03/compi1e_01.html
This *Compile Time* article describes how to deal with deadlock, using the Dining Philosophers example. The beginning of the article also reviews mutual exclusion primitives (e.g., semaphores) and concurrent programming.

Summary

One problem that arises in multiprogrammed systems is deadlock. A process or thread is in a state of deadlock (or is deadlocked) if the process or thread is waiting for a particular event that will not occur. In a system deadlock, one or more processes are deadlocked. Most deadlocks develop because of the normal contention for dedicated resources (i.e., resources that may be used by only one user at a time). Circular wait is characteristic of deadlocked systems.

One example of a system that is prone to deadlock is a spooling system. A common solution is to restrain the input spoolers so that, when the spooling files begin to reach some saturation threshold, they do not read in more print jobs. Today's systems allow printing to begin before the job is completed so that a full, or nearly full, spooling file can be emptied or partially cleared even while a job is still executing. This concept has been applied to streaming audio and video clips, where the audio and video begin to play before the clips are fully downloaded.

In any system that keeps processes waiting while it makes resource-allocation and process scheduling decisions, it is possible to delay indefinitely the scheduling of a process while other processes receive the system's attention. This situation, variously called indefinite postponement, indefinite blocking, or starvation, can be as devastating as deadlock. Indefinite postponement may occur because of biases in a system's resource scheduling policies. Some systems prevent indefinite postponement by increasing a process's priority as it waits for a resource—this technique is called aging.

Resources can be preemptible (e.g., processors and main memory), meaning that they can be removed from a process without loss of work, or nonpreemptible meaning that they (e.g., tape drives and optical scanners), cannot be removed from the processes to which they are assigned. Data and programs certainly are resources that the operating system must control and allocate. Code that cannot be changed while in use is said to be reentrant. Code that may be changed but is reinitialized each time it is used is said to be serially reusable. Reentrant code may be shared by several processes simultaneously, whereas serially reusable code may be used by only one process at a time. When we call particular resources shared, we must be careful to state whether they may be used by several processes simultaneously or by only one of several processes at a time. The latter kind—serially reusable resources —are the ones that tend to become involved in deadlocks.

The four necessary conditions for deadlock are: a resource may be acquired exclusively by only one process at a time (mutual exclusion condition); a process that has acquired an exclusive resource may hold it while waiting to obtain other resources (wait-for condition, also called the hold-and-wait condition); once a process has obtained a resource, the system cannot remove the resource from the process's control until the process has finished using the resource (no-preemption condition); and two or more processes are locked in a "circular chain" in which each process in the chain is waiting for one or more resources that the next process in the chain is holding (circular-wait condition). Because these are necessary conditions for a deadlock to exist, the existence of a deadlock implies that each of them must be in effect. Taken together, all four conditions are necessary and sufficient for deadlock to exist (i.e., if all these conditions are in place, the system is deadlocked).

The four major areas of interest in deadlock research are deadlock prevention, deadlock avoidance, deadlock detection, and deadlock recovery. In deadlock prevention our concern is to condition a system to remove any possibility of deadlocks occurring. Havender observed that a deadlock cannot occur if a system denies any of the four necessary conditions. The first necessary condition, namely that processes claim exclusive use of the resources they require, is not one that we want to break, because we specifically want to allow dedicated (i.e., serially reusable) resources. Denying the "wait-for" condition requires that all of the resources a process needs to complete its task be requested at once, which can result in substantial resource underutilization and raises concerns over how to charge for resources. Denying the "no-preemption" condition can be costly, because processes lose work when their resources are preempted. Denying the "circular-wait" condition uses a linear ordering of resources to prevent deadlock. This strategy can increase efficiency over the other strategies, but not without difficulties.

In deadlock avoidance the goal is to impose less stringent conditions than in deadlock prevention in an attempt to get better resource utilization. Avoidance methods allow the possibility of deadlock to loom, but whenever a deadlock is approached, it is carefully sidestepped. Dijkstra's Banker's Algorithm is an example of a deadlock avoidance algorithm. In the Banker's Algorithm, the system ensures that a process's maximum resource need does not exceed the number of available resources. The system is said to be in a safe state if the operating system can guarantee that all current processes can complete their work within a finite time. If not, then the system is said to be in an unsafe state.

Dijkstra's Banker's Algorithm requires that resources be allocated to processes only when the allocations result in safe states. It has a number of weaknesses (such as requiring a fixed number of processes and resources) that prevent it from being implemented in real systems.

Deadlock detection methods are used in systems in which deadlocks can occur. The goal is to determine if a deadlock has occurred, and to identify those processes and resources involved in the deadlock. Deadlock detection algorithms can incur significant runtime overhead. To facilitate the detection of deadlocks, a directed graph indicates resource allocations and requests. Deadlock can be detected using graph reductions. If a process's resource requests may be granted, then we say that a graph may be reduced by that process. If a graph can be reduced by all its processes, then there is no deadlock. If a graph cannot be reduced by all its processes, then the irreducible processes constitute the set of deadlocked processes in the graph.

Deadlock recovery methods are used to clear deadlocks from a system so that it may operate free of the deadlocks, and so that the deadlocked processes may complete their execution and free their resources. Recovery typically requires that one or more of the deadlocked processes be flushed from the system. The suspend/resume mechanism allows the system to put a temporary hold on a process (temporarily preempting its resources), and, when it is safe to do so, resume the held process without loss of work. Checkpoint/rollback facilitates suspend/resume capabilities by limiting the loss of work to the time at which the last checkpoint (i.e., saved state of the system) was taken. When a process in a system terminates (by accident or intentionally as the result of a deadlock recovery algorithm), the system performs a rollback by undoing every operation related to the terminated process that occurred since the last checkpoint. To ensure that data in the database remains in a consistent state when deadlocked processes are terminated, database systems typically perform resource allocations using transactions.

In personal computer systems and workstations, deadlock has generally been viewed as a limited annoyance. Some systems implement the basic deadlock prevention methods suggested by Havender, while others ignore the problem—these methods seem to be satisfactory. While ignoring deadlocks may seem dangerous, this approach can actually be rather efficient. If deadlock is rare, then the processor time devoted to checking for deadlocks significantly reduces system performance. However, given current trends, deadlock will continue to be an important area of research as the number of concurrent operations and number of resources become large, increasing the likelihood of deadlock in multiprocessor and distributed systems. Also, many real-time systems, which are becoming increasingly prevalent, require deadlock-free resource allocation.

Key Terms

Advanced Configuration and Power Interface (ACPI)—Power management specification supported by many operating systems that allows a system to turn off some or all of its devices without loss of work.

aging—Method of preventing indefinite postponement by increasing a process's priority gradually as it waits.

checkpoint—Record of a state of a system so that it can be restored later if a process must be prematurely terminated (e.g., to perform deadlock recovery).

checkpoint/rollback—Method of deadlock and system recovery that undoes every action (or transaction) of a terminated process since the process's last checkpoint.

circular wait—Condition for deadlock that occurs when two or more processes are locked in a "circular chain," in which each process in the chain is waiting for one or more resources that the next process in the chain is holding.

circular-wait necessary condition for deadlock—One of the four necessary conditions for deadlock; states that if a deadlock exists, there will be two or more processes in a circular chain such that each process is waiting for a resource held by the next process in the chain.

deadline scheduling—Scheduling a process or thread to complete by a definite time; the priority of the process or thread may need to be increased as its completion deadline approaches.

deadlock—Situation in which a process or thread is waiting for an event that will never occur.

deadlock avoidance—Strategy that eliminates deadlock by allowing a system to approach deadlock, but ensuring that deadlock never occurs. Avoidance algorithms can achieve higher performance than deadlock prevention algorithms. (See also Dijktra's Banker's Algorithm.)

deadlock detection—Process of determining whether or not a system is deadlocked. Once detected, a deadlock can be removed from a system, typically resulting in loss of work.

deadlock prevention—Process of disallowing deadlock by eliminating one of the four necessary conditions for deadlock.

deadlock recovery—Process of removing a deadlock from a system. This can involve suspending a process temporarily (and preserving its work) or sometimes killing a process (thereby losing its work) and restarting it.

deadly embrace—See deadlock.

dedicated resource—Resource that may be used by only one process at a time. Also known as a serially reusable resource.

Dijkstra's Banker's Algorithm—Deadlock avoidance algorithm that controls resource allocation based on the amount of resources owned by the system, the amount of resources owned by each process and the maximum amount of resources that the process will request during execution. Allows resources to be assigned to processes only when the allocation results in a safe state. (See also safe state and unsafe state.)

Dining Philosophers—Classic problem introduced by Dijkstra that illustrates the problems inherent in concurrent programming, including deadlock and indefinite postponement. The problem requires the programmer to ensure that a set of *n* philosophers at a table containing *n* forks, who alternate between eating and thinking, do not starve while attempting to acquire the two adjacent forks necessary to eat.

graph reduction—Altering a resource-allocation graph by removing a process if that process can complete. This also involves removing any arrows leading to the process (from the resources allocated to the process) or away from the process (to resources the process is requesting). A resource-allocation graph can be reduced by a process if all of that process's resource requests can be granted, enabling that process to run to completion and free its resources.

Havender's linear ordering—See linear ordering.

linear ordering (Havender)—Logical arrangement of resources that requires that processes request resources in a linear order. This method denies the circular-wait necessary condition for deadlock.

maximum need (Dijkstra's Banker's Algorithm) —Characteristic of a process in Dijkstra's Banker's Algorithm that describes the largest number of resources (of a particular type) the process will need during execution.

mutual exclusion necessary condition for deadlock—One of the four necessary conditions for deadlock; states that deadlock can occur only if processes cannot claim exclusive use of their resources.

necessary condition for deadlock—Condition that must be true for deadlock to occur. The four necessary conditions are the mutual exclusion condition, no-preemption condition, wait-for condition and circular-wait condition.

no-preemption necessary condition for deadlock—One of the four necessary conditions for deadlock; states that deadlock can occur only if resources cannot be forcibly removed from processes.

nonpreemptible resource—Resource that cannot be forcibly removed from a process, e.g., a tape drive. Such resources are the kind that can become involved in deadlock.

preemptible resource—Resource that may be removed from a process such as a processor or memory. Such resources cannot be involved in deadlock.

reentrant code—Code that cannot be changed while in use and therefore can be shared among processes and threads.

resource allocation graph—Graph that shows processes and resources in a system. An arrow pointing from a process to a resource indicates that the process is requesting the resource. An arrow pointing from a resource to a process indicates that the resource is allocated to the process. Such a graph helps determine if a deadlock exists and. if so, helps identify the processes and resources involved in the deadlock.

resource type —Grouping of resources that perform a common task.

safe state—State of a system in Dijkstra's Banker's Algorithm in which there exists a sequence of actions that will allow every process in the system to finish without the system becoming deadlocked.

saturation threshold—A level of resource utilization above which the resource will refuse access. Designed to reduce deadlock, it also reduces throughput.

serially reusable code—Code that can be modified but is reinitialized each time it is used. Such code can be used by only one process or thread at a time.

serially reusable resource—See dedicated resource.

shared resource—Resource that can be accessed by more than one process.

starvation—Situation in which a thread waits for an event that might never occur, also called indefinite postponement.

sufficient conditions for deadlock—The four conditions-mutual exclusion, no-preemption, wait-for and circular-wait-which are necessary and sufficient for deadlock.

suspend/resume—Method of halting a process, saving its state, releasing its resources to other processes, then restoring its resources after the other processes have released them.

transaction—Atomic, mutually exclusive operation that either completes or is rolled back. Modifications to database entries are often performed as transactions to enable high performance and reduce the cost of deadlock recovery.

unsafe state—State of a system in Dijkstra's Banker's Algorithm that might eventually lead to deadlock because there might not be enough resources to allow any process to finish.

wait-for condition—One of the four necessary conditions for deadlock; states that deadlock can occur only if a process is allowed to wait for a resource while it holds another.

Exercises

7.1 Define deadlock.

7.2 Give an example of a deadlock involving only a single process and a single resource.

7.3 Give an example of a simple resource deadlock involving three processes and three resources. Draw the appropriate resource-allocation graph.

7.4 What is indefinite postponement? How does indefinite postponement differ from deadlock? How is indefinite postponement similar to deadlock?

7.5 Suppose a system allows for indefinite postponement of certain entities. How would you as a systems designer provide a means for preventing indefinite postponement?

7.6 Discuss the consequences of indefinite postponement in each of the following types of systems.

a. batch processing

b. timesharing

c. real-time

7.7 A system requires that arriving processes must wait for service if the needed resource is busy. The system does not use aging to elevate the priorities of waiting processes to prevent indefinite postponement. What other means might the system use to prevent indefinite postponement?

7.8 In a system of n processes, a subset of m of these processes is currently suffering indefinite postponement. Is it possible for the system to determine which processes are being indefinitely postponed?

7.9 *(The Dining Philosophers)* One of Dijkstra's more delightful contributions is his problem of the Dining Philosophers.[53, 54] It illustrates many of the subtle problems inherent in concurrent programming.

Your goal is to devise a concurrent program (with a monitor) that simulates the behavior of the philosophers. Your program should be free of deadlock and indefinite postponement—otherwise one or more philosophers might starve. Your program must, of course, enforce mutual exclusion—two philosophers cannot use the same fork at once.

Figure 7.12 shows the behavior of a typical philosopher. Comment on each of the following implementations of a typical philosopher.

a. See Fig. 7.13.

b. See Fig. 7.14.

c. See Fig. 7.15.

d. See Fig. 7.16.

```
1  typical Philosopher()
2  {
3     while ( true )
4     {
5        think();
6        eat();
7     } // end while
8  } // end typical Philosopher
```

Figure 7.12 | Typical philosopher behavior for Exercise 7.9.

```
1   typicalPhilosopher()
2   {
3      while ( true )
4      {
```

***Figure 7.13** | Philosopher behavior for Exercise 7.9(a) (Part 1 of 2.)*

```
5          think();
6
7          pickUpLeftFork();
8          pickUpRightFork();
9
10         eat();
11
12         putDownLeftFork();
13         putDownRightFork();
14      } // end while
15
16   } // end typical Philosopher
```

Figure 7.13 | *Philosopher behavior for Exercise 7.9(a). (Part 2 of 2.)*

```
1    typicalPhilosopher()
2    {
3       while ( true )
4       {
5          think();
6
7          pickUpBothForksAtOnce();
8
9          eat();
10
11         putDownBothForksAtOnce();
12      } // end while
13
14   } // end typical Philosopher
```

Figure 7.14 | Phjlosopher behavior for Exercise 7.9(b).

```
1    typicalPhilosopher()
2    {
3       while ( true )
4       {
5          think();
6
7          while ( notHoldingBothForks )
8          {
9             pickUpLeftFork();
10
11            if ( rightForkNotAvailable )
12            {
13               putDownLeftFork();
```

Figure 7.15 | Philosopher behavior for Exercise 7.9(c). (Part 1 of 2.)

```
14            } // end if
15            else
16            {
17               pickUpRightFork();
18            } // end while
19         } // end else
20
21         eat();
22
23         putDownLeftFork() ;
24         putDownRightFork();
25      } // end while
26
27   } // end typical Philosopher
```

Figure 7.15 | Philosopher behavior for Exercise 7.9(c). (Part 2 of 2.)

```
1   typicalPhilosopher()
2   {
3      while ( true )
4      {
5         think();
6
7         if ( philosopherID mod 2 == 0 )
8         {
9            pickUpLeftFork();
10           pickUpRightFork();
11
12           eat();
13
14           putDownLeftFork();
15           putDownRightFork();
16        } // end if
17        else
18        {
19           pickUpRightFork();
20           pickUpLeftFork();
21
22           eat();
23
24           putDownRightFork();
25           putDownLeftFork();
26        } /,/ end else
27     } // end while
28
29  } // end typical Philosopher
```

Figure 7.16 | Philosopher behavior for Exercise 7.9(d).

7.10 Define and discuss each of the following resource concepts.

a. preemptible resource
b. nonpreemptible resource
c. shared resource
d. dedicated resource
e. reentrant code
f. serially reusable code
g. dynamic resource allocation

7.11 State the four necessary conditions for a deadlock to exist. Give a brief intuitive argument for the necessity of each individual condition.

7.12 In the context of the traffic deadlock, illustrated in Fig. 7.1, discuss each of the necessary conditions for deadlock.

7.13 What are the four areas of deadlock research mentioned in the text? Discuss each briefly.

7.14 Havender's method for denying the "wait-for" condition requires that processes must request all of the resources they will need before the system may let them proceed. The system grants resources on an "all-or-none" basis. Discuss the pros and cons of this method.

7.15 Why is Havender's method for denying the "no-preemption" condition not a popular means for preventing deadlock?

7.16 Discuss the pros and cons of Havender's method for denying the "circular-wait" condition.

7.17 How does Havender's linear ordering for denying the "circular-wait" condition prevent cycles from developing in resource-allocation graphs?

7.18 A process repeatedly requests and releases resources of types R_1 and R_2, one at a time and in that order. There is exactly one resource of each type. A second process also requests and releases these resources one at a time repeatedly. Under what circumstances could these processes deadlock? If so, what could be done to prevent deadlock?

7.19 Explain the intuitive appeal of deadlock avoidance over deadlock prevention.

7.20 In the context of Dijkstra's Banker's Algorithm discuss whether each of the states described in Fig. 7.17 and Fig. 7.18 is safe or unsafe. If a state is safe, show how it is possible for all processes to complete. If a state is unsafe, show how it is possible for deadlock to occur.

7.21 The fact that a state is unsafe does not necessarily imply that the system will deadlock. Explain why this is true. Give an example of an unsafe state and show how all of the processes could complete without a deadlock occurring.

7.22 Dijkstra's Banker's Algorithm has a number of weaknesses that preclude its effective use in real systems. Comment on why each of the following restrictions may be considered a weakness in the Banker's Algorithm.

a. The number of resources to be allocated remains fixed.
b. The population of processes remains fixed.
c. The operating system guarantees that resource requests will be serviced in a finite time.
d. Users guarantee that they will return held resources within a finite time.
e. Users must state maximum resource needs in advance.

7.23 *(Banker's Algorithm for Multiple Resource Types)* Consider Dijkstra's Banker's Algorithm as discussed in Section 7.8, Deadlock Avoidance with Dijkstra's Banker's Algorithm. Suppose that a system using this deadlock avoidance scheme has n processes and m different resource types; assume that multiple resources of each type may exist and that the number of resources of each type is known. Develop a version of the Banker's Algorithm that will enable such a system to avoid deadlock. *[Hint:* Under what circumstances could a particular process be guaranteed to complete its execution, and thus return its resources to the pool?]

7.24 Could the Banker's Algorithm still function properly if resources could be requested in groups? Explain your answer carefully.

Process	*max(* P_i *)*	*loan(* P_i *)*	*claim(* P_i *)*
P_1	4	1	3
P_2	6	4	2
P_3	8	5	3
P_4	2	0	2
		$a = 1$	

Figure 7.17 | Resource description for State A.

Process	*max(* P_i *)*	*loan(* P_i *)*	*claim(* P_i *)*
P_1	8	4	4
p_2	8	3	5
P_3	8	5	3
		a = 2	

Figure 7.18 | Resource description for State B.

7.25 A system that uses Banker's-Algorithm deadlock avoidance has five processes (1, 2, 3, 4, and 5) and uses resources of four different types (A, B, C, and D). There are multiple resources of each type. Is the state of the system depicted by Fig. 7.19 and Fig. 7.20 safe? Explain your answer. If the system is safe, show how all the processes could complete their execution successfully. If the system is unsafe, show how deadlock might occur.

7.26 Suppose a system with *n* processes and *m* identical resources uses Banker's Algorithm deadlock avoidance. Write a function boolean isSafeStatel(int[][] maximumNeed, int[][] loans. int[] available) that determines whether the system is in a safe state.

7.27 Suppose a system uses the Banker's Algorithm with *n* processes, *m* resource types, and multiple resources of each type. Write a function boolean isSafeState2 (int[] [] maximumNeed, int[] [] loans, int[] available) that determines whether the system is in a safe state.

7.28 In a system in which it is possible for a deadlock to occur, under what circumstances would you use a deadlock detection algorithm?

7.29 In the deadlock detection algorithm employing the technique of graph reductions, show that the order of the graph reductions does not matter, the same final state will result. [*Hint:* No matter what the order, after each reduction, the available resource pool increases.]

7.30 Why is deadlock recovery such a difficult problem?

7.31 Why is it difficult to choose which processes to "flush" in deadlock recovery?

7.32 One method of recovering from deadlock is to kill the lowest-priority process (or processes) involved in the deadlock. This (these) process(es) could then be restarted and once again allowed to compete for resources. What potential problem might develop in a system using such an algorithm? How would you solve this problem?

Process	*Current loan*				*Maximum*				*Need*				*Current Claim*			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	C
1	1	0	2	0	3	2	4	2	2	2	2	2	2	2	2	2
2	0	3	1	2	3	5	1	2	3	2	0	0	3	2	0	0
3	2	4	5	1	2	7	7	5	0	3	2	4	0	3	2	4
4	3	0	0	6	5	5	0	8	2	5	0	2	2	5	0	2
5	4	2	1	3	6	2	1	4	2	0	0	1	2	0	0	1

Figure 7.19 | System state describing current loan, maximum need and current claim.

Total Resources				*Resources Available*			
A	B	C	D	A	B	C	D
13	13	9	13	3	4	0	1

***Figure 7.20** | System state describing total number of resources and available resources.*

7.33 Why will deadlock probably be a more critical problem in future operating systems than it is today?

7.34 Do resource malfunctions that render a resource unusable generally increase or decrease the likelihood of deadlocks and indefinite postponement? Explain your answer.

7.35 The vast majority of computer systems in use today do allow at least some kinds of deadlock and indefinite postponement situations to develop, and many of these systems provide no automatic means of detecting and recovering from these problems. In fact, many designers believe that it is virtually impossible to certify a system as absolutely free of the possibilities of deadlock and indefinite postponement. Indicate how these observations should affect the design of "mission-critical" systems.

7.36 The table in Fig. 7.21 shows a system in an unsafe state. Explain how all of the processes may manage to finish execution without the system becoming deadlocked.

7.37 A system has three processes and four identical resources. Each process requires at most two of the resources at any given time.

a. Can deadlock occur in this system? Explain.
b. If there are *m* processes, and each could request up to *n* resources, how many resources must be available in the system to ensure that deadlock will never occur?
c. If there are *m* processes and *r* resources in the system, what maximum number of resources, *n,* could each process request, if all processes must have the same maximum?

Suggested Projects

7.38 Prepare a research paper on how current operating systems deal with deadlock.

7.39 Research how real-time systems ensure that deadlock never occurs. How do they manage to eliminate deadlock yet maintain performance?

7.40 Determine how Web servers and other business-critical systems address the problem of deadlock.

Suggested Simulations

7.41 *(Deadlock Detection and Recovery Project)* Write a simulation program to determine whether a deadlock has occurred in a system of *n* identical resources and *m* processes. Have each process generate a set of resources that it wants (e.g., 3 of resource A, 1 of resource B and 5 of resource C). Then, from each set, request the resources one type at a time in a random order and with random pauses between types. Have each process hold onto all the resources that it has acquired until it can get all of them. Deadlocks should start developing. Now have another thread check for deadlocks every few seconds. It should report when a deadlock has occurred and start killing threads involved in the deadlock. Try different heuristics for choosing processes to kill and see which type of heuristic results in the best average time between deadlocks.

7.42 *(Deadlock Prevention Simulation Project)* Write a simulation program to compare various deadlock prevention schemes discussed in Section 7.7, Deadlock Prevention. In particular, compare deadlock prevention by denying the "wait-for" condition (Section 7.7.1, Denying the "Wait-For" Condition) with deadlock prevention by denying the "no-preemption" condition (Section 7.7.2, Denying the "No-Preemption" Condition). Your program should generate a sample user population, user arrival times, user resource needs (assume the system has *n* identical resources) in terms both of maximum needs and of when the resources are actually required, and so on. Each simulation should accumulate statistics on job turnaround times, resource utilization, number of jobs progressing at once (assume that jobs may progress when they have the portion of the *n* resources they currently need), and the like. Observe the results of your simulations and draw conclusions about the relative effectiveness of these deadlock prevention schemes.

Process	*loan(P_i)*	*max(P_i)*	*claim(P_i)*
P_1	1	5	4
P_2	1	3	2
P_3	5	10	5
	a = 1		

Figure 7.21 | *Example of a system in an unsafe state.*

7.43 *(Deadlock Avoidance Simulation Project)* Write a simulation program to examine the performance of a system of *n* identical resources and *m* processes operating under banker's-algorithm resource allocation. Model your program after the one you developed in Exercise 7.42. Run your simulation and compare the results with those observed in your deadlock prevention simulations. Draw conclusions about the relative effectiveness of deadlock avoidance vs. the deadlock prevention schemes studied.

7.44 *(Deadlock Prevention and Avoidance Comparison)* Create a program that simulates arriving jobs with various resource needs (listing each resource that will be needed and the time at which it will be needed). This can be a random-number-based driver program. Use your simulation to determine how deadlock prevention and deadlock avoidance strategies yield higher resource utilization.

Recommended Reading

Though many operating systems simply ignore the problem of deadlock, there has been considerable research in the field to create effective algorithms to solve the problem. Dijkstra, Isloor and Marsland[55] and Zobel[56] characterize the problem of deadlock. Dijkstra was one of the first to document the problem of deadlock in multiprogrammed systems, presenting both the Dining Philosophers problem and the banker's algorithm for deadlock avoidance.[57, 58] Coffman, Elphick and Shoshani later provided a framework for research in this area by classifying the four necessary conditions for deadlock.[59] Holt[60, 61] developed resource-allocation graphs to aid in deadlock pre--ention, while Habermann[62] presented a family of deadlock avoidance techniques. In discussing the UNIX System V operating system, Bach indicates how deadlocks may develop in areas such as interrupt handling.[63] Kenah et al. discuss the detection and handling of deadlocks in Digital Equipment Corporation's VAX/VMS operating system.[64]

Today's systems rarely implement deadlock prevention or avoidance mechanisms. In fact, in distributed systems, conventional deadlock prevention and avoidance mechanisms are not feasible, because each computer in a distributed system may be managed by a different operating system. Researchers have developed deadlock detection mechanisms[65] coupled with checkpoint/rollback capabilities[66] to reduce loss of work. Research into deadlock is currently focused on developing efficient deadlock detection algorithms and applying them to distributed systems, networks and other deadlock-prone environments.[67] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Isloor, S. S., and T. A. Marsland, "The Deadlock Problem: An Overview," *Computer,* Vol. 13, No. 9, September 1980, pp. 58-78.
2. Zobel, D., "The Deadlock Problem: A Classifying Bibliography," *Operating Systems Review,* Vol. 17, No. 4, October 1983, pp. 6-16.
3. Holt, R. G, "Some Deadlock Properties of Computer Systems," *ACM Computing Surveys,Vol.* 4, No. 3, September 1972, pp. 179-196.
4. Coffman, E. G., Jr.; Elphick, M. J.; and A. Shoshani, "System Deadlocks," *Computing Surveys,Vol.* 3,No. 2, June 1971,p. 69.
5. Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in F. Genuys, ed., *Programming Languages,* New York: Academic Press, 1968.
6. Dijkstra, E. W., "Hierarchical Ordering of Sequential Processes," *Acta Informatica,Vol.* 1,1971, pp. 115-138.
7. Coffman, E. G, Jr.; Elphick, M. J.; and A. Shoshani, "System Deadlocks," *Computing Surveys,* Vol. 3, No. 2, June 1971, pp. 67-78.
8. Habermann, A. N., "Prevention of System Deadlocks," *Communications of the ACM,Vol.* 12, No. 7, July 1969, pp. 373-377,385.
9. Holt, R. C, "Comments on the Prevention of System Deadlocks," *Communications of the ACM,* Vol. 14, No. 1, January 1971, pp. 36-38.
10. Holt, R. C, "On Deadlock Prevention in Computer Systems," Ph.D. Thesis, Ithaca, NY.: Cornell University, 1971.
11. Parnas, D. L., and A. N. Haberman, "Comment on Deadlock Prevention Method," *Communications of the ACM,* Vol. 15, No. 9, September 1972, pp. 840-841.
12. Newton, G., "Deadlock Prevention, Detection, and Resolution: An Annotated Bibliography," *ACM Operating Systems Review,* Vol. 13, No. 2, April 1979, pp. 33-44.
13. Gelernter, D., "A DAG Based Algorithm for Prevention of Store-and-Forward Deadlock in Packet Networks," *IEEE Transactions on Computers,Vol* C-30,No. 10, October 1981, pp. 709-715.
14. Havender, J. W., "Avoiding Deadlock in Multitasking Systems," *IBM Systems Journal,* Vol. 7, No. 2,1968, pp. 74-84.

15. Brinch Hansen, P., *Operating System Principles,* Englewood Cliffs, NJ: Prentice Hall, 1973.
16. Scherr, A. L., "Functional Structure of IBM Virtual Storage Operating Systems, Part II: OS/VS2-2 Concepts and Philosophies," *IBM Systems Journal,* Vol. 12, No. 4,1973, pp. 382-400.
17. Auslander, M. A.; Larkin, D. C; and A. L. Scherr, "The Evolution of the MVS Operating System," *IBM Journal of Research and Development,* Vol. 25, No. 5,1981, pp. 471-482.
18. Kenah, L. J.; Goldenberg, R. E.; and S. F. Bate, *VAX/VMS Internals and Data Structures,* Version 4.4, Bedford, MA: Digital Press, 1988.
19. Brinch Hansen, P., *Operating System Principles,* Englewood Cliffs, NJ: Prentice Hall, 1973.
20. Dijkstra, E. W., *Cooperating Sequential Processes,* Technological University, Eindhoven, The Netherlands, 1965.
21. Dijkstra, E. W., *Cooperating Sequential Processes,* Technological University, Eindhover, Netherlands, 1965, Reprinted in F. Genuys, ed., *Programming Languages,* New York: Academic Press, 1968.
22. Habermann, A. N, "Prevention of System Deadlocks," *Communications of the ACM,* Vol. 12, No. 7, July 1969, pp. 373-377,385.
23. Madduri, H, and R. Finkel, "Extension of the Banker's Algorithm for Resource Allocation in a Distributed Operating System," *Information Processing Letters,* Vol. 19, No. 1, July 1984, pp. 1-8.
24. Havender, J. W., "Avoiding Deadlock in Multitasking Systems," *IBM Systems Journal.Vol. 1,* No. 2,1968, pp. 74-84.
25. Fontao, R. O., "A Concurrent Algorithm for Avoiding Deadlocks," *Proc. Third ACM Symposium on Operating Systems Principles,* October 1971, pp. 72-79.
26. Frailey, D. J., "A Practical Approach to Managing Resources and Avoiding Deadlock," *Communications of the ACM,* Vol. 16, No. 5, May 1973, pp. 323-329.
27. Devillers, R., "Game Interpretation of the Deadlock Avoidance Problem," *Communications of the ACM,* Vol. 20, No. 10, October 1977, pp. 741-745.
28. Lomet, D. B., "Subsystems of Processes with Deadlock Avoidance," *IEEE Transactions on Software Engineering,* Vol. SE-6, No. 3, May 1980, pp. 297-304.
29. Merlin, P. M., and P. J. Schweitzer, "Deadlock Avoidance in Store-and-Forward Networks—I: Store and Forward Deadlock," *IEEE Transactions on Communications,* Vol. COM-28, No. 3, March 1980, pp. 345-354.
30. Merlin, P. M., and P. J. Schweitzer, "Deadlock Avoidance in Store-and-Forward Networks—II: Other Deadlock Types," *IEEE Transactions on Communications,* Vol. COM-28, No. 3, March 1980, pp. 355-360.
31. Minoura, T, "Deadlock Avoidance Revisited," *Journal of the ACM,* Vol. 29, No. 4, October 1982, pp. 1023-1048.
32. Murphy, J. E., "Resource Allocation with Interlock Detection in a Multitask System," *AFIPS FJCC Proc.,* Vol. 33, No. 2,1968, pp. 1169-1176.
33. Newton, G,, "Deadlock Prevention, Detection, and Resolution: An Annotated Bibliography," *ACM Operating Systems Review,* Vol. 13, No. 2, April 1979, pp. 33-44.
34. Gligor, V., and S. Shattuch, "On Deadlock Detection in Distributed Systems," *IEEE Transactions on Software Engineering,* Vol. SE-6, No. 5, September 1980, pp. 435-440.
35. Ho, G. S., and C. V. Ramamoorthy, "Protocols for Deadlock Detection in Distributed Database Systems," *IEEE Transactions on Software Engineering,* Vol. SE-8, No. 6, November 1982, pp. 554-557.
36. Obermarck, R., "Distributed Deadlock Detection Algorithm." *ACM Transactions on Database Systems,Vol. 1,* No. 2, June 1982. pp. 187-208.
37. Chandy, K. M., and J. Misra, "Distributed Deadlock Detection." *ACM Transactions on Computer Systems,Vol.* 1, No. 2, May 1983. pp. 144-156.
38. Jagannathan, J. R., and R. Vasudevan, "Comments on 'Protocols for Deadlock Detection in Distributed Database Systems'." *IEEE Transactions on Software Engineering,* Vol. SE-9, No. 3. May 1983, p. 371.
39. Kenah, L. I; Goldenberg, R. E.; and S. F. Bate, *VAX/VMS Internals and Data Structures,* version 4.4, Bedford, MA: Digital Press, 1988.
40. Pun H. Shiu, YuDong Tan, Vincent J. Mooney, "A Novel Parallel Deadlock Detection Algorithm and Architecture" *Proceedings of the Ninth International Symposium on Hardware/Software Codesign,* April 2001.
41. Holt, R. C, "Some Deadlock Properties of Computer Systems." *ACM Computing* Surveys, Vol. 4, No. 3, September 1972, pp. 179-196.
42. Holt, R. C, "Some Deadlock Properties of Computer Systems." *ACM Computing Surveys,* Vol. 4, No. 3, September 1972, pp. 179-196.
43. Kenah, L. J.; R. E. Goldenberg; and S. F. Bate, *VAX/VMS Internals and Data Structures,* version 4.4, Bedford, MA: Digital Press, 1988.
44. Thomas, K., *Programming Locking Applications,* IBM Corporation, 2001, <www-124.ibm.com/developerworks/oss/dlm/currentbook/dlmbook_index.html>.
45. Compaq Computer Corporation, Intel Corporation, Microsoft Corporation, Phoenix Technologies Ltd., Toshiba Corporation. "Advanced Configuration and Power Management," rev. 2.0b, October 11,2002, p. 238
46. "Driver Development Tools: Windows DDK, Deadlock Detection," *Microsoft MSDN Library,* June 6, 2003. <msdn.microsoft.com/library/en-us/ddtools/hh/ddtools/dv_8pkj.asp>.
47. "Kernel-Mode Driver Architecture: Windows DDK, Preventing Errors and Deadlocks While Using Spin Locks," *Microsoft MSDN Library,* June 6, 2003, <msdn.microsoft.com/library en-us/kmarch/hh/kmarch/synchro_5ktj.asp>.
48. Krivokapic, N; Kemper, A.; and E. Gudes, "Deadlock Detection in Distributed Database Systems: A New Algorithm and Com-

parative Performance Analysis," *The VLDB Journal— The International Journal on Very Large Data* Bases, Vol. 8, No. 2,1999, pp. 79-100.

49. Krivokapic, N.; Kemper, A.; and E. Gudes, "Deadlock Detection in Distributed Database Systems: A New Algorithm and Comparative Performance Analysis," *The VLDB Journal— The International Journal on Very Large Data Bases,* Vol. 8, No. 2,1999, pp. 79-100.
50. Krivokapic, N.; Kemper, A.; and E. Gudes, "Deadlock Detection in Distributed Database Systems: A New Algorithm and Comparative Performance Analysis," *The VLDB Journal— The International Journal on Very Large Data Bases,* Vol. 8, No. 2,1999, pp. 79-100.
51. Magarshack, P., and P. Paulin, "System-on-chip Beyond the Nanometer Wall," *Proceedings of the 40th Conference on Design Automation,* Anaheim, CA: ACM Press, 2003, pp. 419-424.
52. Benini, L.; Macci, A.; and M. Poncino, "Energy-Aware Design of Embedded Memories: A Survey of Technologies, Architectures, and Techniques," *ACM Transactions on Embedded Computer Systems (TECS),* Vol. 2, No. 1,2003, pp. 5-32.
53. Dijkstra, E. W., "Solution of a Problem in Concurrent Programming Control," *Communications of the ACM,* Vol. 8, No. 5, September 1965, p. 569.
54. Dijkstra, E. W., "Hierarchical Ordering of Sequential Processes," *Acta Informatica,Vol.* 1,1971, pp. 115-138.
55. Isloor, S. S., and T. A. Marsland, "The Deadlock Problem: An Overview," *Computer,* Vol. 13, No. 9, September 1980, pp. 58-78.
56. Zobel, D, "The Deadlock Problem: A Classifying Bibliography," *Operating Systems Review,* Vol. 17, No. 4, October 1983, pp. 6-16.
57. Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in E Genuys, ed., *Programming Languages,* New York: Academic Press, 1968.
58. Dijkstra, E. W., "Hierarchical Ordering of Sequential Processes," *Acta Informatica,* Vol. 1,1971, pp. 115-138.
59. Coffman, E. G., Jr.; Elphick, M. J.; and A. Shoshani, "System Deadlocks," *Computing Surveys,* Vol. 3, No. 2, June 1971, pp. 67-78.
60. Holt, R. C, "Comments on the Prevention of System Deadlocks," *Communications of the ACM,* Vol. 14, No. 1, January 1971, pp. 36-38.
61. Holt, R. G, "On Deadlock Prevention in Computer Systems," Ph.D. Thesis, Ithaca, NY.: Cornell University, 1971.
62. Habermann, A. N., "Prevention of System Deadlocks," *Communications of the ACM,* Vol. 12, No. 7, July 1969, pp. 373-377,385.
63. Bach, M. J., *The Design of the UNIX Operating System,* Englewood Cliffs, NJ: Prentice Hall, 1986.
64. Kenah, L. J.; Goldenberg, R. E.; and S. F. Bate, *VAX/VMS Internals and Data Structures,* version 4.4, Bedford, MA: Digital Press, 1988.
65. Obermarck, R., "Distributed Deadlock Detection Algorithm," *ACM Transactions on Database Systems,* Vol. 7, No. 2, June 1982, pp. 187-208.
66. Krivokapic, N.; Kemper, A.; and E. Gudes, "Deadlock Detection in Distributed Database Systems: A New Algorithm and Comparative Performance Analysis," *The VLDB lournal— The International lournal on Very Large Data Bases,* Vol. 8, No. 2,1999, pp. 79-100.
67. Shiu, P. H.; Tan, Y; and V J. Mooney, "A Novel Parallel Deadlock Detection Algorithm and Architecture," *Proceedings of the Ninth International Symposium on Hardware/Software Codesign,* April 2001.

The heavens themselves, the planets and this center observe degree, priority, and place, ...

—William Shakespeare—

Nothing in progression can, rest on its original plan. We may as well think of rocking a grows man in the cradle of an infant.

—Edmund Burke—

For every problen there is one solution which is simple, neat, and wrong.

—H. L. Mencken—

There is nothing more requisite in business than dispatch.

—Joseph Addison—

Chapter 8

Processor Scheduling

Objectives

After reading this chapter, you should understand:

- *the goals of processor scheduling.*
- *preemptive vs. nonpreemptive scheduling.*
- *the role of priorities in scheduling.*
- *scheduling criteria.*
- *common scheduling algorithms.*
- *the notions of deadline scheduling and real-time scheduling.*
- *Java thread scheduling.*

Chapter Outline

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

8.1 Introduction

We have discussed how multiprogramming enables an operating system to use its resources more efficiently. When a system has a choice of processes to execute, it must have a strategy —called a **processor scheduling policy** (or **discipline)** —for deciding which process to run at a given time. A scheduling policy should attempt to satisfy certain performance criteria, such as maximizing the number of processes that complete per unit time (i.e., throughput), minimizing the time each process waits before executing (i.e., latency), preventing indefinite postponement of processes, ensuring that each process completes before its stated deadline, or maximizing processor utilization. Some of these goals, such as maximizing processor utilization and throughput, are complementary; others conflict with one another —a system that ensures that processes will complete before their deadlines may not achieve the highest throughput, fn this chapter, we discuss the problems of determining when processors should be assigned, and to which processes. Although we focus on processes, many of the topics we describe apply to jobs and threads as well.

Self Review

1. When might a system that ensures that processes will complete before their deadlines not achieve the highest throughput?
2. Which performance criterion is the most important in an operating system? Why is this a hard question to answer?

Ans: **1)** This occurs, for example, when several short processes are delayed while the system dispatches a long process that must meets its deadline. 2) No one performance criterion is more important than the others for every operating system. It depends on the goals of the system. For example, in real-time systems, giving processes and threads immediate, predictable service is more important than high processor utilization. In supercomputers that perform lengthy calculations, processor utilization is typically more important than minimizing latency.

8.2 Scheduling levels

In this section we consider three levels of scheduling (Fig. 8.1). **High-level scheduling—also** called **job scheduling** or **long-term scheduling—**determines which jobs the system allows to compete actively for system resources. This level is sometimes called **admission scheduling,** because it determines which jobs gain admission to the system. Once admitted, jobs are initiated and become processes or groups of processes. The high-level scheduling policy dictates the degree **of multiprogramming—** the total number of processes in a system at a given time.[1] Entry of too many processes into the system can saturate the system's resources, leading to poor performance. In this case, the high-level scheduling policy may decide to temporarily prohibit new jobs from entering until other jobs complete.

After the high-level scheduling policy has admitted a job (which may contain one or more processes) to the system, the **intermediate-level scheduling** policy determines which processes shall be allowed to compete for processors. This policy

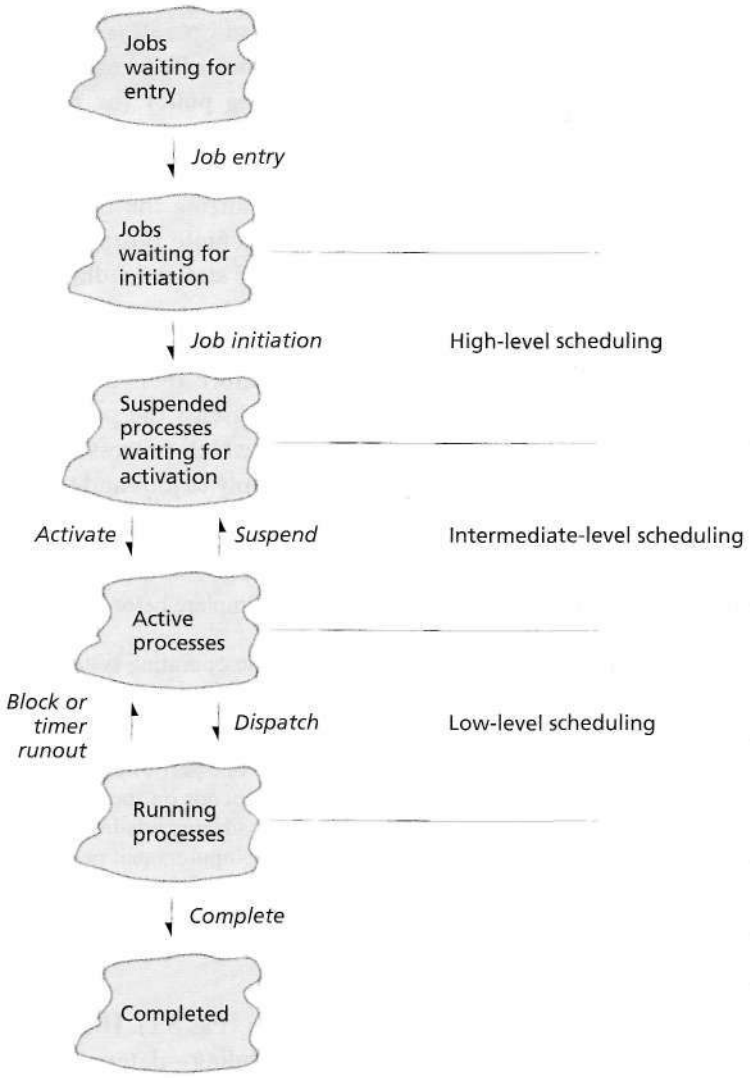

Figure 8.1 | Scheduling levels.

responds to short-term fluctuations in system load. It temporarily suspends and resumes processes to achieve smooth system operation and to help realize certain systemwide performance goals. The intermediate-level scheduler acts as a buffer between the admission of jobs to the system and the assignment of processors to the processes representing these jobs.

A system's **low-level scheduling** policy determines which active process the system will assign to a processor when one next becomes available. In many of today's systems, the low- and intermediate-level schedulers are the only schedulers.

(In this case, job initiation is performed by the intermediate-level scheduler.) High-level schedulers are often limited to large mainframe systems that perform batch processing.

Low-level scheduling policies often assign a **priority** to each process, reflecting its importance—the more important a process, the more likely the scheduling policy is to select it to execute next. We discuss priorities in Section 8.4, Priorities, and throughout this chapter. The low-level scheduler (also called the **dispatcher)** also assigns (i.e., **dispatches)** a processor to the selected process. The dispatcher operates many times per second and must therefore reside in main memory at all times.

In this chapter we discuss many low-level scheduling policies. We present each in the context of certain scheduling objectives and criteria (which we discuss in Section 8.5, Scheduling Objectives, and Section 8.6, Scheduling Criteria), and we describe how they relate to one another. Coffman and Kleinrock discuss popular scheduling policies and indicate how users who know which one the system employs can actually achieve better performance by taking appropriate measures.[2] Ruschitzka and Fabry give a classification of scheduling algorithms, and they formalize the notion of priority.[3]

Self Review

1. How should the intermediate scheduler respond to fluctuations in system load?
2. Which level of scheduler should remain resident in main memory? Why?

Ans: **1)** The intermediate scheduler can prohibit processes from proceeding to the low-level scheduler when the system becomes overloaded and can allow these processes to proceed when the system load returns to normal. **2)** The low-level scheduler should remain resident in main memory because it executes frequently, requiring it to respond quickly to reduce scheduling overhead.

8.3 Preemptive vs. Nonpreemtpive Scheduling

Scheduling disciplines are either preemptive or nonpreemptive. A scheduling discipline is **nonpreemptive** if, once the system has assigned a processor to a process, the system cannot remove that processor from that process. A scheduling discipline is **preemptive** if the system can remove the processor from the process it is running. Under a nonpreemptive scheduling discipline, each process, once given a processor, runs to completion or until it voluntarily relinquishes its processor. Under a preemptive scheduling discipline, the processor may execute a portion of a process's code and then perform a context switch.

Preemptive scheduling is useful in systems in which high-priority processes require rapid response. In real-time systems (discussed in Section 8.9, Real-Time Scheduling), for example, the consequences of not responding to an interrupt could be catastrophic.[4, 5, 6, 7] In interactive timesharing systems, preemptive scheduling helps guarantee acceptable user response times. Preemption is not without cost-context switches incur overhead (see the Operating Systems Thinking feature,

Overhead). To make preemption effective, the system must maintain many processes in main memory, so that the next process is ready when a processor becomes available. As we will see in Chapter 10, Virtual Memory Organization, only a portion of each process typically is in main memory at any time; the less active portions typically reside on disk.

In nonpreemptive systems, short processes can experience lengthy service delays while longer processes complete, but turnaround times are more predictable, because incoming high-priority processes cannot displace waiting processes. Because a nonpreemptive system cannot remove a process from a processor until it completes, errant programs that never complete (e.g., by entering an infinite loop) may never relinquish control of the system. Also, in a nonpreemptive system, executing unimportant processes can make important processes wait.

To prevent users from monopolizing the system (either maliciously or accidentally), a preemptive system can take the processor away from a process. As discussed in Chapter 3, Process Concepts, this is typically implemented by setting an interrupting clock or interval timer that periodically generates an interrupt, which allows the operating system to execute. Once a processor is assigned to a process, the process executes until it voluntarily releases its processor, or until the clock interrupt or some other interrupt occurs. The operating system may then decide whether the running process should continue or some other "next" process should execute.

The interrupting clock helps guarantee reasonable response times to interactive users, prevents the system from getting hung up on a user in an infinite loop.

Operating Systems Thinking

Overhead

Ultimately, computer systems exist to run applications for users. Although operating systems certainly perform important tasks, they consume valuable system resources in doing so; that resource consumption is said to be overhead, because the resources are not directly used by user applications to perform useful work. Operating systems designers seek to minimize that overhead while maximizing the portion of the systems resources that can be allocated to user applications. As we will see, overhead can improve performance by improving resource utilization; it can also reduce performance to provide a higher level of protection and security. As computer power continues to increase with costs decreasing, the resource consumption lost to overhead can become less of an issue. However, designers should be aware of a system's workload when considering overhead. For example, a large overhead for a small load may become relatively small under a heavy load; a small overhead for a small load may become relatively large under a heavy load.

and allows processes to respond to time-dependent events. Processes that need to run periodically depend on the interrupting clock.

In designing a preemptive scheduling mechanism, one must carefully consider the arbitrariness of priority schemes. It does not make sense to build a sophisticated mechanism to faithfully implement a priority preemption scheme when the priorities themselves are not meaningfully assigned.

SelfReview

1. When is nonpreemptive scheduling more appropriate than preemptive scheduling?
2. Can a program that enters an infinite loop monopolize a preemptive system?

Ans: **1)** Nonpreemptive scheduling provides predictable turnaround times, which is important for batch processing systems that must provide users with accurate job-completion times. **2)** This depends on the priority of the process and the scheduling policy. In general, a preemptive system containing a process executing an infinite loop will experience reduced throughput but will still be able to execute other processes periodically. A high-priority process that enters an infinite loop, however, may execute indefinitely if all other processes in the system have a lower priority. In general, preemptive systems are less affected by such programs than nonpreemptive systems. Operating systems typically deal with such situations by limiting the maximum time a process can use a processor.

8.4 Priorities

Schedulers often use priorities to determine how to schedule and dispatch processes. Priorities may be statically assigned or may change dynamically. Priorities quantify the relative importance of processes.

Static priorities remain fixed, so static-priority-based mechanisms are relatively easy to implement and incur relatively low overhead. Such mechanisms are not, however, responsive to changes in environment, even those that could increase throughput and reduce latency.

Dynamic priority mechanisms are responsive to change. For example, the system may want to increase the priority of a process that holds a key resource needed by a higher-priority process. After the first process relinquishes the resource, the system lowers the priority, so that the higher-priority process may execute. Dynamic priority schemes are more complex to implement and have greater overhead than static schemes. Hopefully, the overhead is justified by the increased responsiveness of the system.

In multiuser systems, an operating system must provide reasonable service to a large community of users but should also provide for situations in which a member of the user community needs special treatment. A user with an important job may be willing to pay a premium, i.e., to **purchase priority,** for a higher level of service. This extra charge is merited because resources may need to be withdrawn from other paying customers. If there were no extra charge, then all users would request the higher level of service.

Self Review

1. Why is it worthwhile to incur the higher cost and increased overhead of a dynamic priority mechanism?
2. Why would a dynamic scheduler choose to favor a low-priority process requesting an underutilized resource?

Ans: **1)** A carefully designed dynamic priority mechanism could yield a more responsive system than would a static priority mechanism. **2)** The underutilized resource is likely to be available, allowing the low-priority process to complete and exit the system sooner than a higher-priority process waiting for a saturated resource.

8.5 Scheduling Objectives

A system designer must consider a variety of factors when developing a scheduling discipline, such as the type of system and the users' needs. For example, the scheduling discipline for a real-time system should differ from that for an interactive desktop system; users expect different results from these kinds of systems. Depending on the system, the user and designer might expect the scheduler to:

- *Maximize throughput. A* scheduling discipline should attempt to service the maximum number of processes per unit time.
- *Maximize the number of interactive processes receiving "acceptable" response times.*
- *Maximize resource utilization.* The scheduling mechanisms should keep the resources of the system busy.
- *Avoid indefinite postponement. A* process should not experience an unbounded wait time before or while receiving service.
- *Enforce priorities.* If the system assigns priorities to processes, the scheduling mechanism should favor the higher-priority processes.
- *Minimize overhead.* Interestingly, this is not generally considered to be one of the most important objectives. Overhead often results in wasted resources. But a certain portion of system resources effectively invested as overhead can greatly improve overall system performance.
- *Ensure predictability.* By minimizing the statistical variance in process response times, a system can guarantee that processes will receive predictable service levels (see the Operating Systems Thinking feature, Predictability).

A system can accomplish these goals in several ways. In some cases, the scheduler can prevent indefinite postponement of processes through aging—gradually increasing a process's priority as the process waits for service. Eventually, its priority becomes high enough that the scheduler selects that process to run.

The scheduler can increase throughput by favoring processes whose requests can be satisfied quickly, or whose completion frees other processes to run. One such strategy favors processes holding key resources. For example, a low-priority process may be holding a key resource that is required by a higher-priority process. If the

resource is nonpreemptible, then the scheduler should give the low-priority process more execution time than it ordinarily would receive, so that it will release the key resource sooner. This technique is called **priority inversion,** because the relative pri-----es of the two processes are reversed so that the high-priority one will obtain the resource it requires to continue execution. Similarly, the scheduler may choose to favor a process that requests underutilized resources, because the system will be more likely to satisfy this process's requests in a shorter period of time.

Many of these goals conflict with one another, making scheduling a complex problem. For example, the best way to minimize response times is to have sufficient resources available whenever they are needed. The price for this strategy is that overall resource utilization will be poor. In real-time systems, fast, predictable responses are crucial, and resource utilization is less important. In other types of systems, the economics often makes effective resource utilization imperative.

Despite the differences in goals among systems, many scheduling disciplines exhibit similar properties:

- **Fairness.** A scheduling discipline is fair if all similar processes are treated the same, and no process can suffer indefinite postponement due to scheduling issues (see the Operating Systems Thinking feature, Fairness).
- **Predictability.** A given process always should run in about the same amount of time under similar system loads.
- **Scalability.** System performance should degrade gracefully (i.e., it should not immediately collapse) under heavy loads.

Operating Systems Thinking

Predictability

Predictability is as important in your everyday life as it is in computers. What would you do if you stopped for a red traffic light, and the light stayed red for a long time, much longer than any red light delay you had ever experienced. You would probably become annoyed and fidgety, and after waiting for what you thought was a reasonable amount of time; you might be inclined to go through the light, a potentially dangerous action.

Similarly, you have a sense of how long common tasks should take to perform on your computer. It is challenging for operating systerns to ensure predictability, espedaily given that the load on a system can vary considerably based on the system load and the nature of the tasks being performed. Predictability is especially important for interactive users who demand prompt, consistent response times. Predictability is also important for real-time jobs, where human lives may be at stake (we discuss real-time scheduling in Section 8.9, Real-Time Scheduling). We discuss predictability issues throughout the book.

Self Review

1. How do the goals of reducing the variance in response times and enforcing priorities conflict?
2. Is scheduling overhead always "wasteful"?

Ans: **1)** In preemptive systems, high-priority processes can preempt lower-priority ones ai any time, thereby increasing the variance in response times. **2)** No, the overhead incurred bj effective scheduling operations can increase resource utilization.

8.6 Scheduling Criteria

To realize a system's scheduling objectives, the scheduler should consider process behavior. A **processor-bound** process tends to use all the processor time that the system allocates for it. An **I/O-bound** process tends to use the processor only briefly before generating an I/O request and relinquishing it. Processor-bound processes spend most of their time using the processor; I/O-bound processes spend most of their time waiting for external resources (e.g., printers, disk drives, network connections, etc.) to service their requests, and only nominal time using processors.

A scheduling discipline might also consider whether a process is batch or interactive. A **batch process** contains work for the system to perform without interacting with the user. An **interactive process** requires frequent inputs from the user. The system should provide good response times to an interactive process, whereas a batch process generally can suffer reasonable delays. Similarly, a scheduling discipline should be sensitive to the urgency of a process. An overnight batch process

Operating Systems Thinking

Fairness

How many times have you expressed annoyance saying, "That's not fair!" You have probably heard the expression, "Life is not fair." We all know what fairness is and most people would agree that it is a "good thing" but not universally achieved. We discuss fairness issues in many chapters throughout this book. We will see priority-based scheduling strategies that yield good performance for most users, processes, threads, I/O requests and the like, but do so at the expense of other users, processes, threads and I/O requests that wind up being mistreated. "That's not fair," you say, yet it is hard to ignore how these strategies achieve the system's objectives, such as improving overall system performance.

Operating systems must keep fairness in mind, but always in the context of other considerations. Fairness issues are particularly important in designing resource scheduling strategies, as we will see when we investigate processor scheduling in this chapter and disk scheduling in Chapter 12, Disk Performance Optimization.

does not require immediate responses. A real-time process control system that monitors a gasoline refinery must be responsive, possibly to prevent an explosion.

When the second edition of this book was published, users interacted with processes by issuing trivial requests using a keyboard. In this environment, a scheduler could favor an interactive process with little effect on other processes, because the time required to service interactive processes (e.g., by displaying text) was nominal. AS computers became more powerful, system designers and application programmers included features such as graphics and GUIs to improve user-friendliness. Although some systems still use text-based interfaces, most of today's users interact via GUIs, using a mouse to perform actions such as opening, resizing, dragging and closing windows. Users expect systems to respond quickly so that these actions produce smooth movement. Unlike text display, this can be a computationally intensive task requiring the system to redraw the screen many times per second. Favoring these interactive processes can significantly reduce the level of service provided to other processes in the system. In the case of a batch process, this temporary reduction in service may be acceptable, though perhaps not for processes that execute in real time (e.g., multimedia applications).

In a system that employs priorities, the scheduler should favor processes with higher priorities. Schedulers can base their decisions on how frequently a higher priority process has preempted a lower-priority one. Under some disciplines, frequently preempted processes receive less favored treatment. This is because the process's short runtime before preemption does not justify the overhead incurred by a context switch each time the process is dispatched. One can argue to the contrary that such processes should receive more favored treatment to make up for previous "mistreatment."

Preemptive scheduling policies often maintain information about how much real execution time each process has received. Some designers believe a process that has received little execution time should be favored. Others believe a process that has received much execution time could be near completion and should be favored to help it reach completion, free its resources for other processes to use and exit the system as soon as possible. Similarly, a scheduler may maintain an estimate of how much time remains before a process completes. It is easy to prove that average waiting times can be minimized by running those processes first that require the minimum runtime until completion. Unfortunately, a system rarely knows exactly how much more time each process needs to complete.

Self Review

1. Are interactive processes generally processor bound or I/O bound? How about batch processes?
2. The scheduler rarely knows exactly how much more time each process needs to complete. Consider a system that schedules processes based on this estimation. How can processes abuse this policy?

Ans: **1)** Interactive processes often wait for input from users, so they are generally I/O bound, but an interactive process could certainly enter a phase during which it is primarily processor bound. Batch processes do not interact with users and are often processor bound. Batch processes that require frequent access to disk or other I/O devices are I/O bound. **2)** Processes would tend to underestimate their times to completion to receive favored treatment from the scheduler.

8.7 Scheduling Algorithms

In previous sections we discussed scheduling policies, which specify the goal of the scheduler (e.g., maximizing throughput or enforcing priorities). In the subsections that follow we discuss scheduling algorithms that determine at runtime which process runs next. These algorithms decide when and for how long each process runs; they make choices about preemptibility, priorities, running time, time-to-completion, fairness and other process characteristics. As we will see, some systems require the use of a particular type of scheduler (e.g., real-time systems typically require preemptive, priority-based schedulers). Others rely on process behavior when making scheduler decisions (e.g., favoring I/O-bound processes).

8.7.1 First-In-First-Out (FIFO) Scheduling

Perhaps the simplest scheduling algorithm is **first-in-first-out (FIFO),** also called first-come-first-served (FCFS) (Fig. 8.2). Processes are dispatched according to their arrival time at the ready queue. FIFO is nonpreemptive — once a process has a processor, the process runs to completion. FIFO is fair in that it schedules processes according to their arrival times, so all processes are treated equally, but somewhat unfair because long processes make short processes wait, and unimportant processes make important processes wait. FIFO is not useful in scheduling interactive processes because it cannot guarantee short response times.

FIFO is rarely used as a master scheme in today's systems, but it is often found within other schemes. For example, many scheduling schemes dispatch processes according to priority, but processes with the same priority are dispatched in FIFO order.

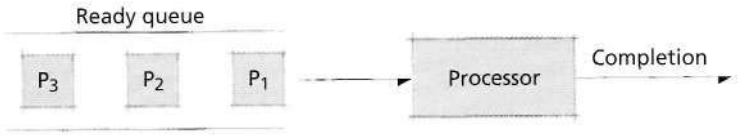

Figure 8.2 | *First-in-first out scheduling.*

Self Review

1. Can indefinite postponement occur in a system that uses a FIFO scheduler? Assume that all processes eventually run to completion (i.e., no process enters an infinite loop).
2. (T/F) FIFO scheduling is rarely found in today's systems.

Ans: **1)** No, indefinite postponement cannot occur, because arriving processes must enter at the back of the queue, meaning they cannot prevent already waiting processes from executing. **2)** False. FIFO can be found within many of today's scheduling algorithms (e.g., priority-based schedulers that dispatch processes with the same priority in FIFO order).

8.7.2 Round-Robin (RR) Scheduling

In **round-robin (RR)** scheduling (Fig. 8.3), processes are dispatched FIFO but are given a limited amount of processor time called a **time slice or** a **quantum.**[8] If a process does not complete before its quantum expires, the system preempts it and gives the processor to the next waiting process. The system then places the preempted process at the back of the ready queue. In Fig. 8.3, process P_1 is dispatched to a processor, where it executes either until completion, in which case it exits the system, or until its time slice expires, at which point it is preempted and placed at the tail of the ready queue. The scheduler then dispatches process P_2.

Round-robin is effective for interactive environments in which the system needs to guarantee reasonable response times. The system can minimize preemption overhead through efficient context-switching mechanisms and by keeping waiting processes in main memory.

Like FIFO, round-robin is commonly found within more sophisticated processor scheduling algorithms but is rarely the master scheme. As we will see throughout this section, many more sophisticated scheduling algorithms degenerate to either FIFO or round-robin when all processes have the same priority. For this reason, FIFO and round-robin are two of the three scheduling algorithms required by the POSIX specification for real-time systems (we discuss real-time scheduling in Section 8.9, Real-Time Scheduling).[9]

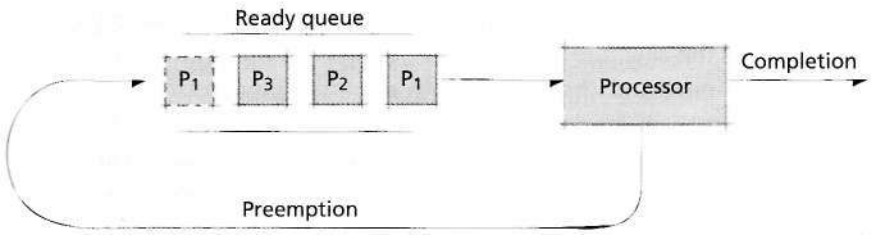

Figure 8.3 | *Round-robin Scheduling.*

Selfish Round-Robin

Kleinrock discussed a variant of round-robin called **selfish round-robin (SRR)** that uses aging to gradually increase process priorities over time.[10] In this scheme, as each process enters the system, it first resides in a holding queue, where it ages until the its priority reaches the level of processes in the active queue. At this point, it is placed in the active queue and scheduled round-robin with other processes in the queue. The scheduler dispatches only processes in the active queue, meaning that older processes are favored over those that have just entered the system.

In SRR. a process's priority increases at a rate a while in the holding queue, and at a rate b, where $b \leq a$, in the active queue. When $b < a$, processes in the holding queue age at a higher rate than those in the active queue, so they will eventually enter the active queue and contend for the processor. Tuning the parameters a and b impacts how a process's age affects average latency and throughput. For example, as a becomes much larger than b, then processes that enter the system will spend little, if any, time in the holding queue. If $b << a$, then processes spend an insignificant amount of time in the holding queue, so SRR degenerates to round-robin. If $b = a$, every process in the system ages at the same rate, so SRR degenerates to FIFO. Exercise 8.23 investigates some properties of the SRR scheme.

Quantum Size

Determination of quantum size, q, is critical to the effective operation of a computer system with preemptive scheduling.[11] Should the quantum be large or small? Should it be fixed or variable? Should it be the same for all processes, or should it be determined separately for each process?

First, let us consider the behavior of the system as the quantum gets either extremely large or extremely small. As the quantum gets large, processes tend to receive as much time as they need to complete, so the round-robin scheme degenerates to FIFO. As the quantum gets small, context-switching overhead dominates; performance eventually degrades to the point that the system spends most of its time context switching with little, if any, useful work accomplished.

Just where between zero and infinity should the quantum be set? Consider the following experiment. Suppose a circular dial is marked with values between $q = 0$ and $q = c$, where c is an extremely large value. We begin with the dial positioned at zero. As we turn the dial, the quantum for the system increases. Assume that the system is operational and there are many interactive processes. As we initially rotate the dial, the quantum sizes are near zero, and the context-switching overhead consumes most of the processor's cycles. The interactive users experience a sluggish system with poor response times. As we increase the quantum, response times improve. The percentage of processor consumed by overhead is small enough that the processes receive some processor service, but response times are still not as fast as each user might prefer.

As we turn the dial more, response times continue to improve. Eventually, we reach a quantum size for which most of the interactive processes receive promt

responses from the system, but it is still not clear if the quantum setting is optimal. We turn the dial a bit further, and response times become slightly better. As we continue to turn the dial, response times start to become sluggish again. The quantum, as it gets larger, eventually becomes large enough for each process to run to completion upon receiving the processor. The scheduling is degenerating to FIFO, in which longer processes make shorter ones wait, and the average waiting time increases as the longer processes run to completion before yielding the processor.

Consider the supposedly optimal value of the quantum that yielded good response times. It is a small fraction of a second. Just what does this quantum represent? It is large enough so that the vast majority of interactive requests require less time than the duration of the quantum. When an interactive process begins executing. it normally uses the processor only briefly—just long enough to generate an I/O request, then block—at which point the process then yields the processor to the next process. The quantum is larger than this compute-until-I/O time. Each time a process obtains the processor, there is great likelihood that the process will run until it generates an I/O request. This maximizes I/O utilization and provides relatively rapid response times for interactive processes. It does so with minimal impact to processor-bound processes, which continue to get the lion's share of processor time because I/O-bound processes block soon after executing.

Just what is the optimal quantum in actual seconds? Clearly, the size varies from system to system and under different loads. It also varies from process to process, but our particular experiment is not geared to measuring differences in processes.

In Linux, the default quantum assigned to a process is 100ms, but can vary from 10 to 200ms, depending on process priority and behavior. High-priority and I/O-bound processes receive a larger quantum than low-priority and processor-bound processes.[12] In Windows XP, the default quantum assigned to a process is an architecture-specific value equalling 20ms on most systems. This value can vary depending on whether the process executes in the foreground or background of the GUI.[13]

When all processes are processor bound, the additional overhead detracts from system performance. However, even when only processor-bound processes are active, preemption still is useful. For example, consider that processor-bound processes could be controlling a real-time, mission critical system—it would be devastating if a process entered an infinite loop or even a phase in which it demanded more processor time than expected. More simply, many processor-bound systems support occasional interactive processes, so preemption is needed to ensure that arriving interactive processes receive good response times.

Self Review

1. Imagine turning the quantum dial for a system that contains only I/O-bound processes. After a point $q = c$, increasing the quantum value results in little, if any, change in system performance. What does point c represent, and why is there no change in system performance when $q > c$?
2. The text describes an optimal quantum value that enables each I/O-bound process to execute just long enough to generate an I/O request, then block. Why is this difficult to implement?

Ans: **1)** This point would be the longest period of computation between I/O requests for any processes in the system. Increasing the value of q past c does not affect any processes. because each process blocks before its quantum expires, so the processes cannot take advantage of the additional processor time allocated to them. **2)** In the general case, it is impossible to predict the path of execution a program will take, meaning that the system cannot accurately determine when a process will generate I/O. Therefore, the optimal quantum size is difficult to determine, because it is different for each process and can vary over time.

8.7.3 Shortest-Process-First (SPF) Scheduling

Shortest-process-first (SPF) is a nonpreemptive scheduling discipline in which the scheduler selects the waiting process with the smallest estimated run-time-to-completion. SPF reduces average waiting time over FIFO.[14] The waiting times, however, have a larger variance (i.e., are more unpredictable) than FIFO, especially for large processes.

SPF favors short processes at the expense of longer ones. Many designers advocate that the shorter the process, the better the service it should receive. Other designers disagree, because this strategy does not incorporate priorities (as measured by the importance of a process). Interactive processes, in particular, tend to be "shorter" than processor-bound processes, so this discipline would seem to still provide good interactive response times. The problem is that it is nonpreemptive, so, in general, arriving interactive processes will not receive prompt service.

SPF selects processes for service in a manner ensuring the next one will complete and leave the system as soon as possible. This tends to reduce the number of waiting processes and also the number of processes waiting behind large processes. As a result, SPF can minimize the average waiting time of processes as they pass through the system.

A key problem with SPF is that it requires precise knowledge of how long a process will run, and this information usually is not available. Therefore, SPF must rely on user- or system-supplied run-time estimates. In production environments where the same processes run regularly, the system may be able to maintain reasonable runtime heuristics. In development environments, however, the user rarely knows how long a process will execute.

Another problem with relying on user process duration estimates is that users may supply small (perhaps inaccurate) estimates so that the system will give their programs higher priority. However, the scheduler can be designed to remove this temptation. For example, if a process runs longer than estimated, the system could terminate it and reduce the priority of that user's other processes, even invoking penalties. A second method is to run the process for the estimated time plus a small percentage extra, then "shelve" it (i.e., preserve it in its current form) so that the system may restart it at a later time.[15]

SPF derives from a discipline called short job first (SJF), which might have worked well scheduling jobs in factories but clearly is inappropriate for low-level scheduling in operating systems. SPF, like FIFO, is nonpreemptive and thus not suitable for environments in which reasonable response times must be guaranteed.

Self Review

1. Why is SPF more desirable than FIFO when system throughput is a primary system objective?
2. Why is SPF inappropriate for low-level scheduling in today's operating systems?

Ans: **1)** SPF reduces average wait times, which increases throughput. **2)** SPF does not provide processes with fast response times, which is essential in today's user-friendly, multiprogrammed, interactive systems.

8.7.4 Highest-Response-Ratio-Next (HRRN) Scheduling

Brinch Hansen developed the **highest-response-ratio-next (HRRN)** policy that corrects some of the weaknesses in SPF, particularly the excessive bias against longer processes and the excessive favoritism toward short processes. HRRN is a nonpreemptive scheduling discipline in which each process's priority is a function not only of its service time but also of its time spent waiting for service.[16] Once a process obtains it, the process runs to completion. HRRN calculates dynamic priorities according to the formula

$$\text{priority} = \frac{\text{time waiting} + \text{service time}}{\text{service time}}$$

Because the service time appears in the denominator, shorter processes receive preference. However, because the waiting time appears in the numerator, longer processes that have been waiting will also be given favorable treatment. This technique is similar to aging and prevents the scheduler from indefinitely postponing processes.

Self Review

1. (T/F) With HRRN scheduling, short processes are always scheduled before long ones.
2. Process P_1 has declared a service time of 5 seconds and has been waiting for 20 seconds. Process P_2 has declared a service time of 3 seconds and has been waiting for 9 seconds. If the system uses HRRN, which process will execute first?

Ans: **1)** False. The longer a process waits, the more likely it will be scheduled before shorter processes. **2)** In this case, process P_1 has a priority of 5 and P_2 has a priority of 4, so the system executes P_1 first.

8.7.5 Shortest-Remaining-Time (SRT) Scheduling

Shortest-remaining-time (SRT) scheduling is the preemptive counterpart of SPF that attempts to increase throughput by servicing small arriving processes. SRT was effective for job-processing systems that received a stream of incoming jobs, but it is no longer useful in most of today's operating systems. In SRT, the scheduler selects the process with the smallest estimated run-time-to-completion. In SPF, once a process begins executing, it runs to completion. In SRT, a newly arriving process with a shorter estimated run-time preempts a running process with a longer run-time-to-completion. Again, SRT requires estimates of future process behavior to be effective, and the designer must account for potential user abuse of this system scheduling strategy.

The algorithm must maintain information about the elapsed service time of the running process and perform occasional preemptions. Newly arriving processes with short run-times execute almost immediately. Longer processes, however, have an even longer mean waiting time and variance of waiting times than in SPF. These factors contribute to a larger overhead in SRT than SPF.

The SRT algorithm offers minimum wait times in theory, but in certain situations, due to preemption overhead, SPF might perform better. For example, consider a system in which a running process is almost complete and a new process with a small estimated service time arrives. Should the running process be preempted? The SRT discipline would perform the preemption, but this may not be the optimal choice. One solution is to guarantee that a running process is no longer preemptible when its remaining run time reaches a low-end threshold.

A similar problem arises when a newly arriving process requires slightly less time to complete than the running process. Although the algorithm would correctly preempt the running process, this may not be the optimal policy. For example, if the preemption overhead is greater than the difference in service times between the two processes, preemption results in poorer performance.

As these examples illustrate, the operating systems designer must carefully weigh the overhead of resource-management mechanisms against the anticipated benefits. Also we see that relatively simple scheduling policies can yield poor performance for subtle reasons.

Self Review

1. Is SRT an effective processor scheduling algorithm for interactive systems?
2. Why is SRT an ineffective scheduling algorithm for real-time processes?

Ans: **1)** At first glance, SRT may seem to be an effective algorithm for interactive processes if the tasks performed before issuing I/O are short in duration. However, SRT determines priority based on the run-time-to-completion, not the run-time-to-I/O. Some interactive processes, such as a shell, execute for the lifetime of the session, which would place the shell at the lowest priority level. **2)** SRT can result in a large variance of response times, whereas real-time processes require a small variance in response times to ensure that they will always complete their tasks within a particular period of time.

8.7.6 Multilevel Feedback Queues

When a process obtains a processor, especially when the process has not as yet had a chance to establish a behavior pattern (e.g., how long it typically runs before generating an I/O request, or which portions of memory the process is currently favoring), the scheduler cannot determine the precise amount of processor time the process will require. I/O-bound processes normally use the processor only briefly before generating an I/O request. Processor-bound processes might use the processor for hours at a time if the system makes it available on a nonpreemptible basis. A scheduling algorithm should typically favor short processes, favor I/O-bound processes to get good I/O device utilization and good interactive response times and should determine the nature of a process as quickly as possible and schedule the process accordingly.

Multilevel feedback queues (Fig. 8.4) help accomplish these goals.[17] A new process enters the queuing network at the tail of the highest queue. The process progresses through that queue in FIFO order until the process obtains a processor. If the process completes its execution, or if it relinquishes the processor to wait for I/O completion or the completion of some other event, exits the queuing network. If a process's quantum expires before the process voluntarily relinquishes the processor, the system places the process at the tail of the next lower-level queue. As long as the process uses the full quantum provided at each level, it continues to move to the tail of the next lower queue. Usually there is some bottom-level queue through which the

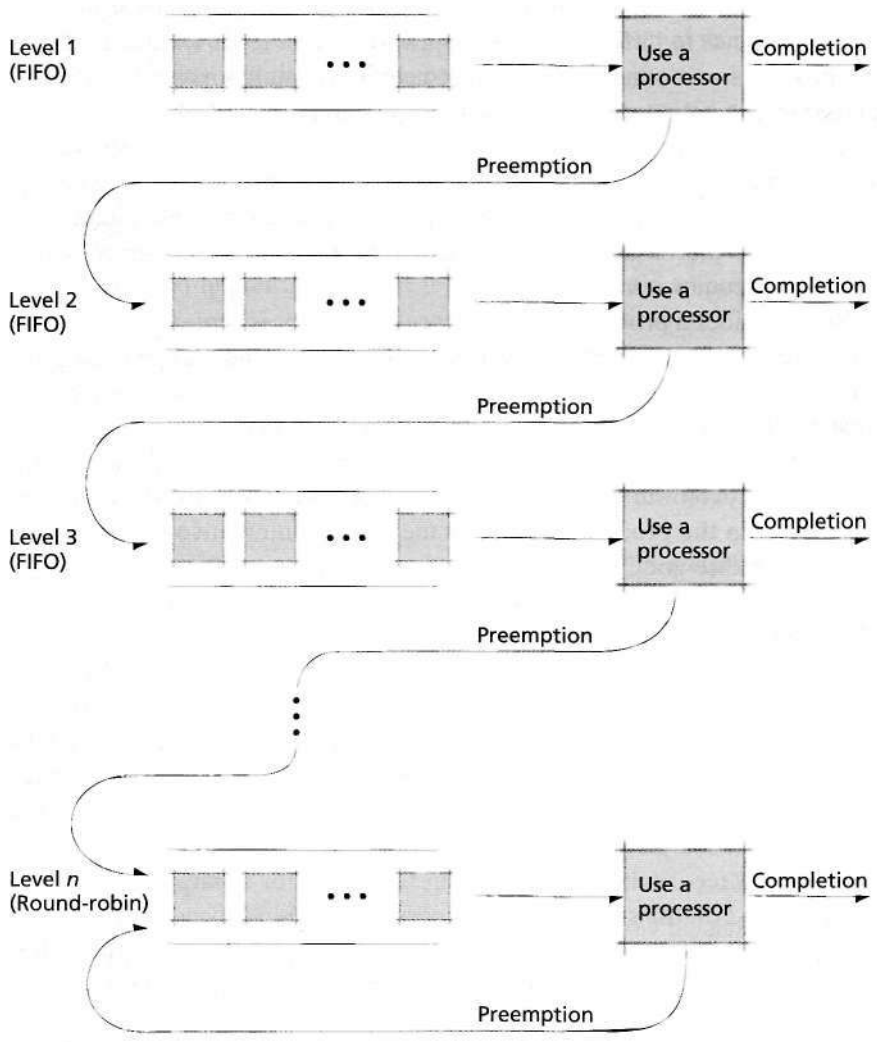

Figure 8.4 | *Multilevel feedback queues.*

process circulates round-robin until it completes. The process that next gets a processor is the one at the head of the highest nonempty queue in the multilevel feedback queue. A running process is preempted by one arriving in a higher queue.

In this system, a process in a lower queue can suffer indefinite postponement, if a higher queue always contains at least one process. This can occur in systems that have a high rate of incoming processes to be serviced, or in which there are several I/O-bound processes consuming their quanta.

In many multilevel feedback schemes, the scheduler increases a process's quantum size as the process moves to each lower-level queue. Thus, the longer a process has been in the queuing network, the larger the quantum it receives each time it obtains the processor. We will soon see why this is appropriate.

Let us examine the treatment that processes receive by considering how this discipline responds to different process types. Multilevel feedback queues favor I/O-bound processes and other processes that need only small bursts of processor time, because they enter the network with high priority and obtain a processor quickly. The discipline chooses for the first queue a quantum large enough so that the vast majority of I/O-bound processes (and interactive processes) issue an I/O request before that quantum expires. When a process requests I/O, it leaves the queuing network, having received the desired favored treatment. The process reenters the network when it next becomes *ready*.

Now consider a processor-bound process. The process enters the network with high priority, and the system places it in the highest-level queue. At this point the queuing network does not "know" whether the process is processor bound or I/O bound—the network's goal is to decide this quickly. The process obtains the processor quickly, uses its full quantum, its quantum expires, and the scheduler moves the process to the next lower queue. Now the process has a lower priority, and incoming processes obtain the processor first. This means that interactive processes will still continue to receive good response times, even as many processor-bound processes sink lower in the queuing network. Eventually, the processor-bound process does obtain the processor, receives a larger quantum than in the highest queue and again uses its full quantum. The scheduler then places the process at the tail of the next-lower queue. The process continues moving to lower queues, waits longer between time slices and uses its full quantum each time it gets the processor (unless preempted by an arriving process). Eventually, the processor-bound process arrives at the lowest-level queue, where it circulates round-robin with other processor-bound processes until it completes.

Multilevel feedback queues are therefore ideal for separating processes into categories based on their need for the processor. When a process exits the queuing network, it can be "stamped" with the identity of the lowest-level queue in which it resided. When the process reenters the queuing network, the system can place it directly in the queue in which it last completed operation—the scheduler here is employing the heuristic that a process's recent-past behavior is a good indicator of its near-future behavior. This technique allows the scheduler to avoid placing a

returning processor-bound process into the higher-level queues, where it would interfere with service to high-priority short processes or to I/O-bound processes.

Unfortunately, if the scheduler always places a returning process in the lowest queue it occupied the last time it was in the system, the scheduler is unable to respond to changes in a process's behavior (e.g., the process may be transitioning from being processor bound to being I/O bound). The scheduler can solve this problem by recording not only the identity of the lowest-level queue in which the process resided, but also the amount of its quantum that was unused during its last execution. If the process consumes its entire quantum, then it is placed in a lower-level queue (if one is available). If the process issues an I/O request before its quantum expires, it can be placed in a higher-level queue. If the process is entering a new phase in which it will change from being processor bound to I/O bound, initially it may experience some sluggishness as the system determines that its nature is changing, but the scheduling algorithm will respond to this change.

Another way to make the system responsive to changes in a process's behavior is to allow the process to move up one level in the feedback queuing network each time it voluntarily relinquishes the processor before its quantum expires. Similarly, the scheduler—when assigning a priority—can consider the time a process has spent waiting for service. The scheduler can age the process by promoting it and placing it in the next higher queue after it has spent a certain amount of time waiting for service.

One common variation of the multilevel feedback queuing mechanism is to have a process circulate round-robin several times through each queue before it moves to the next lower queue. Also, the number of cycles through each queue may be increased as the process moves to the next lower queue. This variation attempts to further refine the service that the scheduler provides to I/O-bound versus processor-bound processes.

Multilevel feedback queuing is a good example of an **adaptive mechanism,** i.e., one that responds to the changing behavior of the system it controls.[18, 19] Adaptive mechanisms generally require more overhead than nonadaptive ones, but the resulting sensitivity to changes in the system makes the system more responsive and helps justify the increased overhead. As we discuss in Section 20.5.2, Process Scheduling, the Linux process scheduler employs an adaptive mechanism borrowing from multilevel feedback queues. *[Note:* In the literature, the terms "process scheduling" and "processor scheduling" have been used equivalently]

SelfReview

1. What scheduling objectives should be evaluated when choosing the number of levels to use in a multilevel feedback queue?
2. Why are adaptive mechanisms desirable in today's schedulers?

Ans: **1)** One major objective is the variance in response times. Increasing the number of levels can cause processor-bound processes to wait longer, which increases the variance in response times. Another objective to consider is resource utilization. As the number of levels

increases, many processes can execute and issue I/Os before their quanta expire, resulting in effective use of both processor time and I/O devices. 2) In today's computers, many applications can be both computationally intensive and I/O bound. For example, a streaming video player must perform I/O to retrieve video clip data from a remote server, then must perform computationally intensive operations to decode and display the video images. Similarly, an interactive game such as a flight simulator must respond to user input (e.g., joystick movements) while rendering complicated 3D scenes. Adaptive mechanisms enable the system to provide alternate responses to these processes as they alternate between I/O-bound and processor-bound behavior.

8.7.7 Fair Share Scheduling

Systems generally support various sets of related processes. For example, UNIX (and other multiuser) systems group processes that belong to an individual user. **Fair share scheduling (FSS)** supports scheduling across such sets of processes.[20, 21, 22, 23] Fair share scheduling enables a system to ensure fairness across groups of processes by restricting each group to a certain subset of the system resources. In the UNIX environment, for example, FSS was developed specifically to "give a pre-specified rate of system resources ... to a related set of users."[24]

Let us consider an example in which fair share scheduling would be useful. Imagine a research group whose members all share one multiuser system. They are divided into two groups. The principal investigators—of which there are few—use the system to perform important, computationally intensive work such as running simulations. The research assistants—of which there are many—use the system for less intensive work such as aggregating data and printing results. Now imagine that many research assistants and only one principal investigator are using the system. The research assistants may consume a majority of the processor time, to the detriment of the principal investigator, who must perform the more important work. However, if the system allowed the research assistants group to use only 25 percent of the processor time and allowed the principal investigators group to use 75 percent, the principal investigator would not suffer such service degradation. In this way, fair share scheduling ensures that the performance of a process is affected only by the population of its process group, and not by the user population as a whole.

Let us investigate how fair share scheduling operates in a UNIX system. Normally, UNIX considers resource-consumption rates across all processes (Fig. 8.5). Under FSS, however, the system apportions the resources to various **fair share groups** (Fig. 8.6). It distributes resources not used by one fair share group to other fair share groups in proportion to their relative needs.

UNIX commands establish fair share groups and associate specific users with them.[25] For the purpose of this discussion, assume that UNIX uses a priority round-robin process scheduler.[26] Each process has a priority, and the scheduler associates processes of a given priority with a priority queue for that value. The process scheduler selects the ready process at the head of the highest-priority queue. Processes

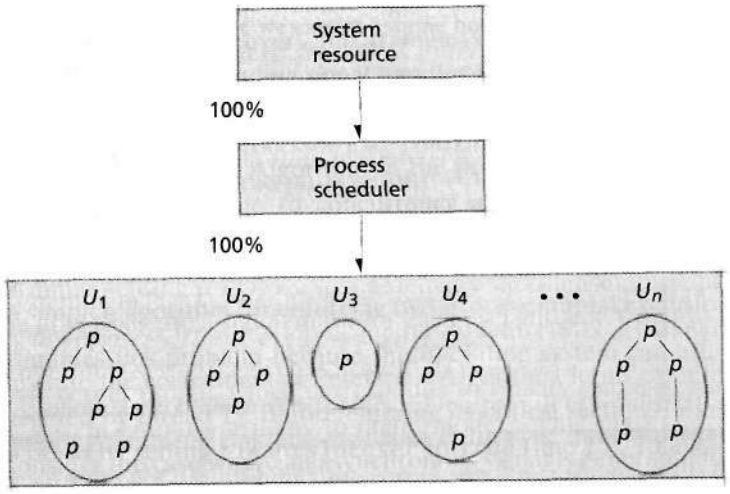

Figure 8.5 *| Standard UNIX process scheduler. The scheduler grants the processor to users, each of whom may have many processes. (Property of AT&T Archives. Reprinted with permission f AT&T)*[27]

within a given priority are scheduled round-robin. A process requiring further service after being preempted receives a lower priority. Kernel priorities are high and apply to processes executing in the kernel; user priorities are lower. Disk events receive higher priority than terminal events. The scheduler assigns the user priority as the ratio of recent processor usage to elapsed real time; the lower the elapsed time, the higher the priority.

The fair share groups are prioritized by how close they are to achieving their specified resource-utilization goals. Groups doing poorly receive higher priority; groups doing well, lower priority.

Self Review

1. In FSS, why should groups that are not achieving their resource-utilization goals be given higher priority?
2. How does FSS differ from standard process scheduling disciplines?

Ans: **1)** Processes that are using fewer resources than specified by their resource-utilization goals are probably suffering from low levels of service. By increasing their priority, the system ensures that these processes execute long enough to use their required resources. **2)** FSS apportions resources to groups of processes, whereas standard process schedulers allow all processes to compete for all resources on an equal footing.

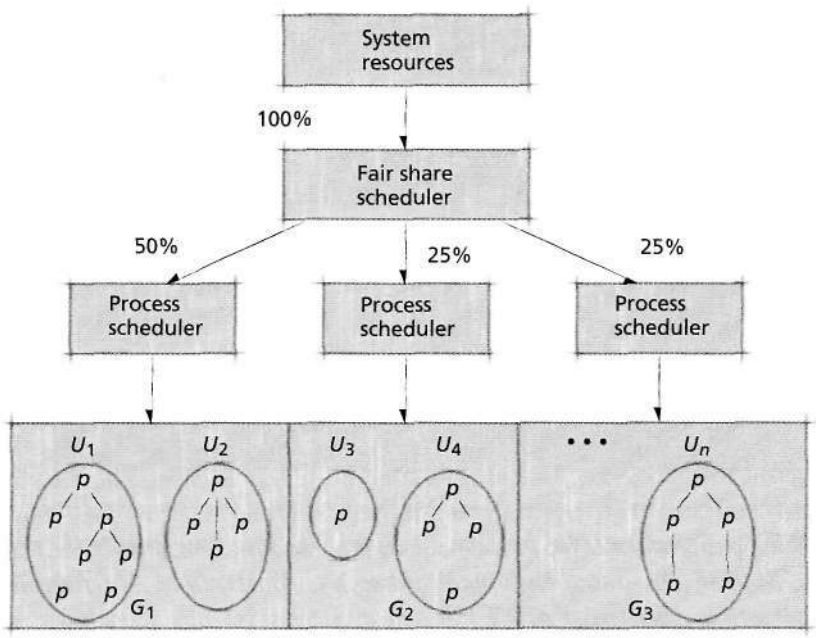

Figure 8.6 *| Fair share scheduler. The fair share scheduler divides system resource capacity into portions, which are then allocated by process schedulers assigned to various fair share groups. (Property of AT&T Archives. Reprinted with permission of AT&T)*[28]

8.8 Deadline Scheduling

In deadline scheduling, certain processes are scheduled to be completed by a specific time or deadline. These processes may have high value if delivered on time and little or no value otherwise.[29, 30, 31]

Deadline scheduling is complex. First, the user must supply the precise resource requirements in advance to ensure that the process is completed by its deadline. Such information is rarely available. Second, the system should execute the deadline process without severely degrading service to other users. Also, the system must carefully plan its resource requirements through to the deadline. This may be difficult, because new processes may arrive, making unpredictable demands. Finally, if many deadline processes are active at once, scheduling could become extremely complex.

The intensive resource management that deadline scheduling requires may generate substantial overhead (see the Operating Systems Thinking feature, Intensity of Resource Management vs. Relative Resource Value). Net consumption of system resources may be high, degrading service to other processes.

As we will see in the next section, deadline scheduling is important to real-time process scheduling. Dertouzos demonstrated that when all processes can meet their deadlines regardless of the order in which they are executed, it is optimal to schedule processes with the earliest deadlines first.[32] However, when the system becomes overloaded, the system must allocate significant processor time to the scheduler to determine the proper order in which to execute processes so that they meet their deadlines. Recent research focuses on the speed[33]-[34] or number[35] of processors the system must allocate to the scheduler so that deadlines are met.

Self Review

1. Why is it important for the user to specify in advance the resources a process needs?
2. Why is it difficult to meet a process's stated deadline?

Ans: **1)** This allows the scheduler to ensure that resources will be available for the process so that it can complete its task before its deadline. **2)** New processes may arrive and place unpredictable demands on the system that prevent it from meeting a process's deadline, resources could fail, the process could dramatically change its behavior and so on.

8.9 Real-Time Scheduling

A primary objective of the scheduling algorithms that we presented in Section 8.7, scheduling Algorithms, is to ensure high resource utilization. Processes that must execute periodically (such as once a minute) require different scheduling algorithms. For example, the unbounded wait times for SPF could be catastrophic for a process that checks the temperature of a nuclear reactor. Similarly, a system using SRT to schedule a process that plays a video clip could produce choppy playback. **Real-time scheduling** meets the needs of processes that must produce correct output by a certain time (i.e., that have a **timing constraint).**[36] A real-time process may divide its instructions into separate tasks, each of which must complete by a dead-

Operating Systems Thinking

Intensity of Resource Management vs. Relative Resource Value

Operating systems manage hardware and software resources. A recurring theme you will see throughout this book is that the intensity with which operating systems need to manage particular resources is proportional to the relative value of those resources, and to their scarcity and intensity of use. For example, the processor and high-speed cache memories are managed much more intensively than secondary storage and other I/O devices. The operating systems designer must be aware of trends that could affect the relative value of system resources and people time and must respond quickly to change.

line. Other real-time processes might perform a certain task periodically, such as updating the locations of planes in an air traffic control system. Real-time schedulers must ensure that timing constraints are met (see the Mini Case Study, Real-Time Operating Systems).

Real-time scheduling disciplines are classified by how well they meet a process's deadlines. **Soft real-time scheduling** ensures that real-time processes are dispatched before other processes in the system, but does not guarantee which, if any, will meet their timing constraints.[37]-[38] Soft real-time scheduling is commonly implemented in personal computers, where smooth playback of multimedia is desirable, but occasional choppiness under heavy system loads is tolerated. Soft real-time systems can benefit from high interrupting clock rates to prevent the system from getting "stuck" executing one process while others miss their deadlines. However, the

Mini Case Study

Real-Time Operating Systems

A real-time system differs from a standard system in that every operation must give results that are both correct and returned within a certain amount of time.[39] Real-time systems are used in time-critical applications such as monitoring sensors. They are often small, embedded systems. Real-time operating systems (RTOSs) must be carefully designed to achieve these goals. (Real-time process scheduling is discussed in Section 8.9, Real-Time Scheduling.)

The control program for the SAGE (Semi-Automatic Ground Environment) project may have been the first real-time operating system.[40,41] SAGE was an Air Force project to defend against possible bomber attacks in the Cold War.[42] Implemented in the end of the 1950s, this system integrated 56 IBM AN/FSQ-7 digital vacuum-tube computers that monitored data from radar systems across the country to track all planes in the United States airspace.[43, 44] It analyzed this information and directed the interceptor fighter planes, thus requiring real-time response.[45] The operating system to handle this task was the largest computer program of its time.[46]

In contrast, today's RTOSs aim for minimal size. There are many available, but QNX and VxWorks lead the field. QNX implements the POSIX standard APIs with its own microkernel and is tailored to embedded systems. It also uses message passing for interprocess communication.[47] Similarly, VxWorks is a microkernel system that complies with the POSIX standard and is widely used in embedded systems.[48] QNX performs better than VxWorks on Intel x86 platforms,[49] but QNX was not available on other processors until version 6.1 (the current version is 6.2).[50] VxWorks, however, has concentrated on the PowerPC processor and has a history of cross-platform compatibility.[51] VxWorks is currently the most common RTOS for embedded systems.[52] Additional RTOSs include Windows CE .NET,[53] OS-9,[54] OSE,[55] and Linux distributions such as uCLinux.[56]

system can incur significant overhead if the interrupt rate is too high, resulting in poor performance or missed deadlines.[57]

Hard real-time scheduling guarantees that a process's timing constraints are always met. Each task specified by a hard real-time process must complete before its deadline; failing to do so could produce catastrophic results, including invalid work, system failure or even harm to the system's users.[58, 59] Hard real-time systems may contain **periodic processes** that perform their computations at regular time intervals (e.g., gathering air traffic control data every second) and **asynchronous** processes that execute in response to events (e.g., responding to high temperatures in a power plant's core).[60]

Static Real-Time Scheduling Algorithms

Static real-time scheduling algorithms do not adjust process priority over time. Because priorities are calculated only once, static real-time scheduling algorithms tend to be simple and incur little overhead. Such algorithms are limited because they cannot adjust to variable process behavior and depend on resources staying up and running to ensure that timing constraints are met.

Hard real-time systems tend to use static scheduling algorithms, because they incur low overhead and it is relatively easy to prove that each process's timing constraints will be met. The **rate-monotonic (RM)** algorithm, for example, is a preemptive, priority-based round-robin algorithm that increases a process's priority linearly (i.e., monotonically) with the frequency (i.e., the rate) with which it must execute. This static scheduling algorithm favors periodic processes that execute frequently.[61] The **deadline rate-monotonic** algorithm can be used when a periodic process specifies a deadline that is not equal to its period.[62]

Dynamic Real-Time Scheduling Algorithms

Dynamic real-time scheduling algorithms schedule processes by adjusting their priorities during execution, which can incur significant overhead. Some algorithms attempt to minimize the scheduling overhead by assigning static priorities to some processes and dynamic priorities to others.[63]

Earliest-deadline-first (EDF) is a preemptive scheduling algorithm that dispatches the process with the earliest deadline. If an arriving process has an earlier deadline than the running process, the system preempts the running process and dispatches the arriving process. The objective is to maximize throughput by satisfying the deadlines of the largest number of processes per unit time (analogous to the SRT algorithm) and minimize the average wait time (which prevents short processes from missing their deadlines while long processes execute). Dertouzos proved that, if a system provides hardware preemption (e.g., timer interrupts) and the real-time processes being scheduled are not interdependent, EDF minimizes the amount of time by which the "most tardy" process misses its deadline.[64] However, many real-time systems do not provide hardware preemption, so other algorithms must be employed.[65]

The **minimum-laxity-first** algorithm is similar to EDF but bases priority on a process's **laxity.** Laxity is a measure of a process's importance based on the amount of time until its deadline and the remaining execution time until its task (which may be periodic) has completed. Laxity is computed using the formula

$$L = D - (T+C),$$

where L is the laxity, D the process's deadline, T the current time and C the process's remaining execution time. For example, if the current time is 5, the deadline for a process is 9 and the process requires 3 units of time to complete, the laxity is 1. If a process has 0 laxity, it must be dispatched immediately or it will miss its deadline. Priorities in minimum-laxity-first are more accurate than those in EDF because they are determined by including the remaining processor time each process requires to complete its task. However, such information is often unavailable.[66]

Self Review

1. Why are most hard real-time scheduling algorithms static?
2. When does the minimum-laxity-first algorithm degenerate to EDF? Can this ever occur?

Ans: **1)** Hard real-time systems must guarantee that processes' deadlines are met. Static scheduling algorithms facilitate proving this property for a particular system and reduce the implementation overhead. **2)** The minimum-laxity-first algorithm degenerates to the EDF algorithm when C is identical for all processes at any given time. It is possible, though highly unlikely, that this would occur.

8.10 Java Thread Scheduling

As discussed in Section 4.6, Threading Models, operating systems provide various levels of support for threads. When scheduling a multithreaded process that implements user-level threads, the operating system is unaware that the process is multithreaded and therefore dispatches the process as one unit, requiring a user-level library to schedule its threads. If the system supports kernel-level threads, it may schedule each thread independently from others within the same process. Still other systems support scheduler activations that assign each process to a kernel-level thread that enables the process's user-level library to perform scheduling operations.

System designers must determine how to allocate quanta to threads, and in what order and with what priorities to schedule threads within a process. For example, a "fair-share" approach divides the quantum allocated to a process among its threads. This prevents a multithreaded process from receiving high levels of service simply by creating a large number of threads. Further, the order in which threads are executed can impact their performance if they rely on one another to continue their tasks. In this section, we present Java thread scheduling. Thread scheduling in Windows XP is discussed in Section 21.6.2, Thread Scheduling. The Linux scheduler (which dispatches both processes and threads) is discussed in Section 20.5.2, Process Scheduling.

One feature of the Java programming language and its virtual machine is that every Java applet or application is multithreaded. Each Java thread is assigned a priority in the range between Thread.MIN_PRIORITY (a constant of 1) and Thread. MAX_PRIORITY (a constant of 10). By default, each thread is given priority Thread. NORM_PRIORITY (a constant of 5). Each new thread inherits the priority of the thread that creates it.

Depending on the platform, Java implements threads in either user or kernel space (see Section 4.6,Threading Models).[67]-[68] When implementing threads in user space, the Java runtime relies on timeslicing to perform preemptive thread scheduling. Without timeslicing, each thread in a set of equal-priority threads runs to completion, unless it leaves the *running* state and enters the *waiting, sleeping* or *blocked* state, or it is preempted by a higher-priority thread. With timeslicing, each thread receives a quantum during which it can execute.

The Java thread scheduler ensures that the highest-priority thread in the Java virtual machine is *running* at all times. If there are multiple threads at the priority level, those threads execute using round-robin. Figure 8.7 illustrates Java's multilevel priority queue for threads. In the figure, assuming a single-processor computer, threads A and B each execute for a quantum at a time in round-robin fashion until both threads complete execution. Next, thread C runs to completion. Threads D, E and F each execute for a quantum in round-robin fashion until they all complete execution. This process continues until all threads run to completion. Note that, depending on the operating system, arriving higher-priority threads could indefinitely postpone the execution of lower-priority ones.

A thread can call the yield method of class Thread to give other threads a chance to execute. Because the operating system preempts the current thread whenever a higher-priority one becomes *ready,* a thread cannot yi el d to a higher-priority thread. Similarly, yield always allows the highest-priority *ready* thread to run, so if all of the *ready* threads are of lower priority than the thread calling yi el d, the current thread will have the highest priority and will continue executing. Therefore, a thread yi el ds to give threads of an equal priority a chance to run. On a timesliced system this is unnecessary, because threads of equal priority will each execute for their quantum (or until they lose the processor for some other reason), and other threads of equal priority will execute round-robin. Thus yi el d is appropriate for nontimesliced systems (e.g., early versions of Solaris[69]), in which a thread would ordinarily run to completion before another thread of equal priority would have an opportunity to run.

1. Why does Java provide the yi el d method? Why would a programmer ever use yield?
2. (T/F) A Java thread of lower priority will never run while a thread of higher priority is *ready.*

Ans: **1)** Method yi el d allows the current thread to voluntarily release the processor and let a thread of equal priority execute. Because Java applications are designed to be portable and because the programmer cannot be certain that a particular platform supports timeslicing,

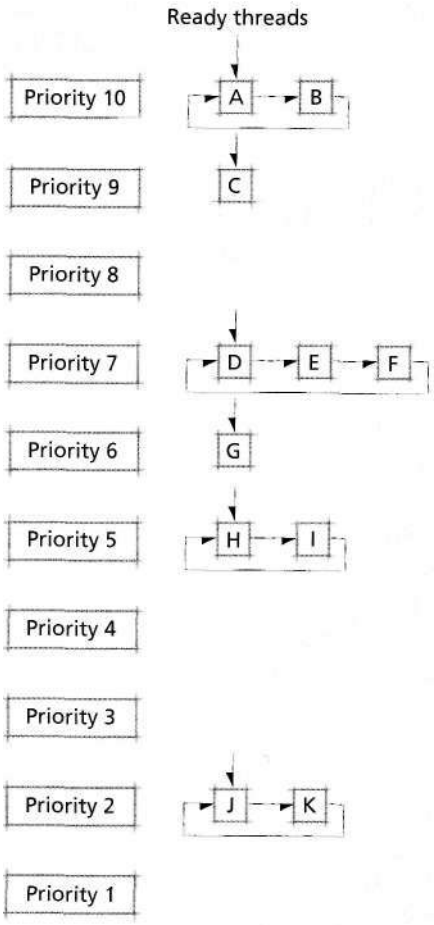

Figure 8.7 | *Java thread priority scheduling.*

programmers use `yield` to ensure that their applications execute properly on all platforms. **2)** True. Java schedulers will execute the highest-priority *ready* thread.

Web Resources

www.linux-tutorial.info/cgi-bin/display.pl?85&0&0&0&3
This portion of the Linux tutorial describes process scheduling in Linux.

developer.apple.com/techpubs/mac/Processes/Processes-16.html
Describes process scheduling for the Apple Macintosh.

www.oreilly.com/catalog/linuxkernel/chapter/ch10.html
Contains the online version of O'Reilly's Linux kernel book This page details process scheduling in Linux version 2.4.

www.javaworld.com/javaworld/jw-07-2002/jw-0703-java01.html
Describes thread scheduling in Java.

csl.cse.ucsc.edu/rt.shtml
Mails the results of the real-time operating system scheduler search unit at the University of California, Santa Cruz.

www.ittc.ku.edu/kurt/
Describes the Kansas University real-time scheduler that uses a modified Linux kernel.

Summary

When a system has a choice of processes to execute, it must have a strategy—called a processor scheduling policy (or discipline)—for deciding which process to run at a given time. High-level scheduling—sometimes called job scheduling or long-term scheduling—determines which jobs the system allows to compete actively for system resources. The high-level scheduling policy dictates the degree of multiprogramming—the total number of processes in a system at a given time. After the high-level scheduling policy has admitted a job (which may contain one or more processes) to the system, the intermediate-level scheduling policy determines which processes shall be allowed to compete for a processor. This policy responds to short-term fluctuations in system load. A system's low-level scheduling policy determines which *ready* process the system will assign to a processor when one next becomes available. Low-level scheduling policies often assign a priority to each process, reflecting its importance—the more important a process, the more likely the scheduling policy is to select it to execute next.

A scheduling discipline can be preemptive or nonpreemptive. To prevent users from monopolizing the system (either maliciously or accidentally), preemptive schedulers set an interrupting clock or interval timer that periodically generates an interrupt. Priorities may be statically assigned or be changed dynamically during the course of execution.

When developing a scheduling discipline, a system designer must consider a variety of objectives, such as the type of system and the users' needs. These objectives may include maximizing throughput, maximizing the number of interactive users receiving "acceptable" response times, maximizing resource utilization, avoiding indefinite postponement, enforcing priorities, minimizing overhead and ensuring predictability of response times. To accomplish these goals, a system can use techniques such as aging processes and favoring processes whose requests can be satisfied quickly. Many scheduling disciplines exhibit fairness, predictability and scalability.

Operating system designers can use system objectives to determine the criteria on which scheduling decisions are made. Perhaps the most important concern is how a process uses the processor (i.e., whether it is processor bound or I/O bound). A scheduling discipline might also consider whether a process is batch or interactive. In a system that employs priorities, the scheduler should favor processes with higher priorities.

Schedulers employ algorithms that make choices about preemptibility, priorities, running time and other process characteristics. FIFO (also called FCFS) is a nonpreemptive algorithm that dispatches processes according to their arrival time at the ready queue. In round-robin (RR) scheduling, processes are dispatched FIFO but are given a limited amount of processor time called a time slice or a quantum. A variant of round-robin called selfish round-robin (SRR) initially places a process in a holding queue until its priority reaches the level of processes in the active queue, at which point, the process is placed in the active queue and scheduled round-robin with others in the queue. Determination of quantum size is critical to the effective operation of a computer system. The "optimal" quantum is large enough so that the vast majority of I/O-bound and interactive requests require less time than the duration of the quantum. The optimal quantum size varies from system to system and under different loads.

Shortest-process-first (SPF) is a nonpreemptive scheduling discipline in which the scheduler selects the waiting process with the smallest estimated run-time-to-completion. SPF reduces average waiting time over FIFO but increases the variance in response times. Shortest-remaining-time (SRT) scheduling is the preemptive counterpart of SPF that selects the process with the smallest estimated run-time-to-completion. The SRT algorithm offers minimum wait times in theory, but in certain situations, due to preemption overhead, SPF might actually perform better. Highest-response-ratio-next (HRRN) is a nonpreemptive scheduling discipline in which the priority of each process is a function not only of its service time but also of the amount of time it has been waiting for service.

Multilevel feedback queues allow a scheduler to dynamically adjust to process behavior. The process that gets a processor next is the one that reaches the head of the highest nonempty queue in the multilevel feedback queuing network. Processor-bound processes are placed at the lowest-level queue, and I/O-bound processes tend to be located in the higher-level queues. A running process is preempted by a process arriving in a higher queue. Multilevel feedback queuing is a good example of an adaptive mechanism.